



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Analisi del Sentiment nelle Comunicazioni Email tramite Intelligenza Artificiale

RELATORE

Prof. Fabio Palomba

Università degli Studi di Salerno

CANDIDATO

Andrea Ruggiero

Matricola: 0512114732

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

"Set your heart ablaze"

Abstract

Con il presente lavoro di tesi si intende sviluppare un modello di predizione del sentimento applicato alle comunicazioni e-mail. L'obiettivo principale è analizzare automaticamente il contenuto testuale dei messaggi e stimarne il grado di negatività, inteso come livello di criticità potenziale associato alla comunicazione.

Non tutte le e-mail presentano infatti lo stesso livello di rilevanza operativa: una comunicazione informativa può risultare neutra o a basso impatto, mentre un messaggio contenente segnalazioni di disservizi, reclami o criticità operative può richiedere un intervento tempestivo. L'automatizzazione di tale processo consente di supportare i meccanismi decisionali, migliorando la capacità di individuare rapidamente comunicazioni potenzialmente problematiche.

Il progetto prevede l'utilizzo di tecniche di Natural Language Processing e modelli di deep learning basati su architetture Transformer pre-addestrate, successivamente sottoposte a fine-tuning su un dataset opportunamente trattato ed etichettato. A differenza dei tradizionali approcci di classificazione discreta, il problema è stato formulato come un task di regressione, con l'obiettivo di predire un punteggio continuo rappresentativo del livello di negatività del messaggio, consentendo una valutazione più granulare e sfumata del contenuto testuale.

Le prestazioni del modello sono state valutate mediante metriche tipiche della regressione, e analizzate attraverso rappresentazioni grafiche. I risultati sono stati inoltre comparati con quelli di un modello alternativo avente la medesima finalità predittiva, al fine di evidenziarne punti di forza e limiti in termini di accuratezza e capacità di generalizzazione. Infine, sono stati proposti possibili sviluppi futuri, orientati all'estensione del modello a scenari applicativi più complessi.

Indice

Elenco delle Figure	iii
Elenco delle Tabelle	iv
1 Introduzione	1
1.1 Contesto del problema	1
1.2 Obiettivi	3
1.2.1 Scelta del modello	3
1.2.2 Costruzione del modello	4
1.3 Risultati attesi	5
1.3.1 Specifiche PEAS	5
1.3.2 Valutazione del modello	8
1.4 Struttura della tesi	8
2 Background	10
2.1 Dataset	10
2.2 Modello	12
2.2.1 Origini del modello	12
2.2.2 BERT	14
2.2.3 Tokenizer	15
2.2.4 Flusso di predizione	17

3	Metodo di ricerca	29
3.1	Sviluppo applicativo	29
3.2	Raffinamento del dataset	30
3.2.1	Pre-processing dei dati	30
3.2.2	Mapping delle etichette in ottica regressiva	31
3.2.3	Bilanciamento del dataset	32
3.3	Suddivisione del dataset	34
3.4	Caricamento modello e tokenizzazione	35
3.5	Addestramento	37
3.5.1	Argomenti del trainer	37
3.5.2	Configurazione degli iperparametri	38
3.5.3	Funzionamento del trainer	41
4	Risultati	46
4.1	Risultati del training	46
4.1.1	Valutazione andamento sul Validation Set	47
4.1.2	Valutazione andamento sul Test Set	48
4.2	Valutazione grafica	49
5	Conclusioni	54
5.1	Comparazione con strumenti analoghi	54
5.1.1	Confronto empirico	55
5.2	Possibili sviluppi futuri	58
5.3	Considerazioni finali	59
	Bibliografia	61

Elenco delle figure

2.1	Illustrazione grafica del flusso di predizione di BERT	28
4.1	Grafico Predicted vs True Values sul Validation set	50
4.2	Grafico Predicted vs True Values sul Test set	50
4.3	Grafico di Distribuzione degli errori sul Validation set	51
4.4	Grafico di Distribuzione degli errori sul Test set	52
4.5	Grafico Residui vs Predizione sul Validation set	53
4.6	Grafico Residui vs Predizione sul Test set	53
5.1	Matrice di confusione delle soglie scelte	56

Elenco delle tabelle

3.1	Tabella di conversione dell'etichetta iniziale del dataset in soglie continue	31
3.2	Censimento del dataset preliminare	32
4.1	Andamento delle metriche di training e validazione per ogni epoca. .	46
4.2	Confronto tra le metriche di Validation e Test Set.	48
5.1	Confronto tra modello LLaMA, modello progettato e valutazione umana	57

CAPITOLO 1

Introduzione

1.1 Contesto del problema

Negli ultimi decenni, la posta elettronica, o e-mail, si è affermata come uno dei principali strumenti di comunicazione digitale a livello globale. Introdotta come sistema per lo scambio di messaggi testuali tra utenti connessi a una rete informatica, l'e-mail ha progressivamente assunto un ruolo centrale nelle dinamiche comunicative, in ambito privato ma, soprattutto, in contesti professionali e aziendali.

Dal punto di vista tecnico, un messaggio di posta elettronica è composto da un'intestazione (header), da un contenuto testuale (body) e da eventuali allegati. Verrà poi inviato tra i server di posta attraverso il protocollo SMTP (Simple Mail Transfer Protocol). Al giorno d'oggi, la trasmissione di informazioni, documenti ufficiali, comunicazioni operative e notifiche automatiche generate da sistemi informatici è gestita quasi totalmente attraverso e-mail.

In ambito aziendale, questo strumento non rappresenta soltanto un mezzo di comunicazione interpersonale, ma costituisce un vero e proprio canale operativo attraverso cui transitano decisioni strategiche, scambi contrattuali, dati sensibili e informazioni riservate.

Esso, dunque, assume una duplice natura: da un lato è uno strumento di collaborazione e coordinamento, dall'altro è un potenziale punto di vulnerabilità del sistema, in quanto l'elevato volume di comunicazioni, unito alla facilità con cui è possibile inviare messaggi senza costi, rende la posta elettronica un canale privilegiato anche per attività fraudolente e malevole. In particolare, un utente si trova a dover fronteggiare problematiche come:

- Phishing e tecniche di social engineering: messaggi di posta elettronica che simulano comunicazioni ufficiali o provenienti da soggetti fidati, con l'obiettivo di indurre l'utente a rivelare credenziali di accesso, informazioni personali o dati sensibili.
- Comunicazioni malevole o sospette: e-mail contenenti notifiche di cambio password non autorizzate, segnalazioni di presunti tentativi di accesso, avvisi di disservizi o richieste urgenti provenienti da fonti non verificate, che possono costituire un tentativo di frode o preludere a un attacco informatico.
- Sovraccarico informativo: condizione derivante dall'elevato volume di messaggi ricevuti quotidianamente, che riduce la capacità cognitiva dell'utente di distinguere tempestivamente le comunicazioni prioritarie da quelle secondarie, aumentando il rischio di trascurare messaggi critici.

Queste criticità hanno un impatto diretto sulla sicurezza, in quanto un singolo messaggio non gestito correttamente può aprire la strada a gravi violazioni, con conseguenze economiche o reputazionali. Una classificazione manuale dei messaggi da parte degli utenti non è sostenibile, né efficiente, soprattutto in realtà complesse con un flusso costante e massivo di comunicazioni. In questo scenario, l'applicazione di tecniche di analisi del sentimento (sentiment analysis) ai messaggi e-mail rappresenta una possibile soluzione innovativa, permettendo di assegnare un livello di criticità ai testi ricevuti e di supportare l'utente nell'individuazione rapida delle comunicazioni più delicate.

1.2 Obiettivi

L'obiettivo è quello di realizzare uno strumento in grado di analizzare le e-mail ricevute dall'utente, stimandone automaticamente il sentiment tramite un modello di elaborazione del linguaggio naturale. In questo contesto, il sentiment rappresenta il livello di criticità potenziale associato al contenuto della comunicazione. Il processo di ricerca è quindi iniziato dalla definizione dell'approccio di modellazione più adatto al problema.

1.2.1 Scelta del modello

Un modello di classificazione che, a primo impatto, sembrerebbe la scelta ottimale per il raggiungimento dello scopo, per sua natura, restituisce come output la classe predetta alla quale un messaggio viene assegnato, accompagnata da una misura di confidenza relativa alla decisione presa. Tuttavia, questo tipo di output non risulta pienamente coerente con gli obiettivi del progetto. Infatti, ciò che interessa non è semplicemente etichettare un'e-mail come positiva o negativa, ma ottenere anche un valore numerico continuo che quantifichi il grado di criticità del messaggio. Questo approccio consente di ordinare le e-mail non solo in base a categorie discrete, ma anche secondo un livello graduato di criticità, portando così all'attenzione dell'utente le comunicazioni più delicate.

Per questo motivo, è stato scelto un approccio basato sulla regressione, in grado di fornire una variabile numerica che rappresenta il livello di severità del messaggio. Successivamente, a ciascun punteggio può essere associata una classe discreta, derivata dal valore numerico tramite soglie definite attraverso un'analisi dei dati. In questo modo, oltre a mantenere una categorizzazione chiara di classi distinte, tipica di un modello di classificazione, si conserva anche una gerarchia interna alla classe discreta, che permette di distinguere i casi più o meno gravi all'interno della stessa fascia di criticità.

1.2.2 Costruzione del modello

Per quanto riguarda la costruzione del modello, si è optato per una strategia di fine-tuning a partire da un modello linguistico pre-addestrato. Il fine-tuning è una tecnica di apprendimento supervisionato in cui un modello pre-addestrato su un ampio corpus generico viene ulteriormente addestrato su un dataset specifico, al fine di adattarlo a un compito particolare. Questa scelta è motivata da diverse considerazioni. Lo sviluppo di un modello da zero richiederebbe grandi quantità di dati annotati, elevate risorse computazionali e tempi di addestramento molto lunghi. D'altra parte, utilizzare un modello generico senza fine-tuning (ad esempio, addestrato su recensioni di film o testi social) potrebbe non garantire prestazioni ottimali, poiché il linguaggio delle e-mail aziendali presenta caratteristiche peculiari, come terminologia tecnica, stile formale e lessico specifico di procedure e sistemi informatici.

Il fine-tuning rappresenta quindi un compromesso ideale, in quanto consente di sfruttare la conoscenza linguistica già appresa dal modello di base (strutture sintattiche e semantiche generali) e di adattarla al dominio specifico delle e-mail aziendali e personali. In tal modo, con un numero relativamente limitato di dati per l'addestramento, è possibile ottenere un modello altamente performante, capace di rilevare le sfumature di criticità presenti nei testi.

L'adozione di un modello di regressione sottoposto a fine-tuning appare dunque la soluzione più adeguata, in quanto bilancia efficienza, accuratezza e adattabilità al contesto, fornendo uno strumento realmente utile per la classificazione critica dei messaggi di posta elettronica.

1.3 Risultati attesi

1.3.1 Specifiche PEAS

Per definire i risultati attesi dallo strumento di analisi del sentiment, è utile fare riferimento alle specifiche PEAS, un metodo consolidato per descrivere formalmente le caratteristiche di un agente intelligente, comprendendo quali siano gli obiettivi principali dell'agente, il contesto in cui esso opera, e gli strumenti a sua disposizione per percepire l'ambiente e agire su di esso. L'acronimo PEAS quindi indica:

- P – Performance measure (misura di prestazione)
- E – Environment (ambiente)
- A – Actuators (attuatori)
- S – Sensors (sensori)

La Misura di prestazione definisce il modo in cui l'efficacia dell'agente sarà valutata. Per il nostro modello, le metriche principali includono:

- **Accuratezza delle previsioni:** Il modello deve classificare correttamente il sentiment delle e-mail, minimizzando l'errore tra il valore predetto e quello reale. Per una valutazione quantitativa si utilizzano metriche tipiche dei problemi di regressione, in quanto il livello di criticità è espresso su scala continua.

In particolare:

- *Errore Medio Assoluto (MAE):*

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

dove y_i rappresenta il valore reale e $\hat{y}_i$ il valore predetto dal modello. Il MAE misura l'errore medio in valore assoluto, fornendo un'indicazione diretta della deviazione media delle previsioni rispetto ai valori reali. È una metrica facilmente interpretabile poiché mantiene la stessa unità di misura della variabile target.

– *Errore Quadratico Medio (MSE)*:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

L'MSE penalizza maggiormente gli errori più grandi, poiché eleva al quadrato la differenza tra valore reale e predetto. Questo rende la metrica più sensibile agli outlier e agli errori significativi, risultando utile quando si desidera evitare predizioni fortemente errate.

L'obiettivo del modello è minimizzare entrambe le metriche, garantendo predizioni quanto più possibile vicine ai valori reali.

- **Capacità di spiegare la varianza:** La qualità del modello viene inoltre valutata tramite il coefficiente di determinazione R^2 , definito come:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

dove $\bar{y}$ rappresenta la media dei valori reali. Il coefficiente R^2 misura la proporzione della varianza totale dei dati che viene spiegata dal modello. Un valore di R^2 prossimo a 1 indica che il modello riesce a spiegare quasi completamente la variabilità del sentiment osservato; un valore vicino a 0 indica invece una capacità predittiva limitata.

- **Robustezza:** il modello deve mantenere performance stabili anche su dati non visti durante il training, riducendo il rischio di sovradattamento (*overfitting*) e garantendo affidabilità nelle condizioni reali.
- **Efficienza:** tempi di inferenza contenuti per consentire un'analisi in tempo reale delle e-mail, fondamentale per applicazioni pratiche in contesti aziendali o personali.
- **Stabilità:** una volta addestrato, il modello deve conservare le proprie capacità predittive fino a un re-train programmato, evitando fluttuazioni non giustificate nei risultati.

L'**ambiente** in cui l'agente opera è definito come segue:

- **Parzialmente osservabile:** il modello può analizzare solo il testo delle e-mail, senza accesso diretto all'intenzione reale dell'autore.
- **Agente singolo:** non vi sono altri agenti con cui interagire; il modello agisce autonomamente.
- **Stocastico:** dati simili possono produrre risultati leggermente diversi a causa di sfumature linguistiche e ambiguità semantiche.
- **Continuo:** l'output è espresso su una scala numerica continua che rappresenta il livello di criticità del sentiment.
- **Statico:** durante l'elaborazione, il testo non subisce modifiche esterne.
- **Episodico:** ogni e-mail viene analizzata indipendentemente dalle altre.

Gli **attuatori** definiscono le modalità attraverso cui l'agente manifesta le proprie decisioni sull'ambiente. Nel presente sistema, l'azione dell'agente consiste nella restituzione di un punteggio numerico (*score*) derivante dall'elaborazione del sentiment per ciascuna e-mail analizzata. A esso sarà poi associata una classe discreta (positiva, negativa o neutra) determinata sulla base di soglie predefinite applicate allo score. In questo modo, il modello fornisce sia un'informazione continua sia una categorizzazione qualitativa immediatamente interpretabile.

I **sensori** rappresentano gli strumenti attraverso cui l'agente percepisce l'ambiente. In questo caso, l'input del sistema è costituito dai dati testuali delle e-mail, in particolare dal corpo del messaggio. Le informazioni verranno fornite al modello, che le elaborerà ai fini di predirne il sentiment.

1.3.2 Valutazione del modello

La valutazione dell'ottimalità del modello sarà effettuata, dunque, attraverso l'analisi delle metriche di performance precedentemente introdotte, ovvero Mean Squared Error (MSE), Mean Absolute Error (MAE) e coefficiente di determinazione R^2 .

Oltre alla valutazione dei risultati ottenuti nella fase di addestramento, il modello verrà sottoposto a test su un dataset separato, non utilizzato durante l'addestramento. Tale procedura consente di misurare in maniera oggettiva la capacità di generalizzazione del sistema su dati completamente nuovi, riducendo il rischio che le prestazioni osservate siano influenzate da fenomeni di overfitting e garantendo il rispetto del requisito di robustezza. Il modello sarà considerato ottimale qualora dimostri prestazioni coerenti e soddisfacenti sia dalle metriche prodotte durante l'addestramento sia nella valutazione sul dataset indipendente.

A partire dai risultati ottenuti in entrambe le fasi, verranno inoltre prodotti e analizzati grafici diagnostici finalizzati a fornire una valutazione visiva del comportamento inferenziale del modello, supportando l'interpretazione quantitativa delle metriche attraverso un'analisi qualitativa delle predizioni.

1.4 Struttura della tesi

La tesi è organizzata in cinque capitoli principali, ciascuno dei quali affronta un aspetto specifico del progetto e del lavoro svolto.

1. **Introduzione**, in questo capitolo vengono presentati il contesto e le motivazioni alla base del lavoro svolto. Si analizza il ruolo della posta elettronica nella comunicazione moderna. Successivamente, vengono illustrati gli obiettivi della tesi, motivando le scelte progettuali adottate. Infine, vengono anticipati i risultati attesi, introducendo le specifiche PEAS del sistema e delineando il tipo di analisi che verrà condotta per la valutazione delle performance del modello.

2. **Background**, questo capitolo fornisce le basi teoriche necessarie alla piena comprensione del progetto. In primo luogo, viene descritto il dataset utilizzato, illustrandone le caratteristiche principali e le motivazioni che ne hanno guidato la selezione. Successivamente, vengono approfondite le caratteristiche del modello BERT adottato, analizzandone l'architettura, il tokenizer con il suo meccanismo di tokenizzazione e il flusso di predizione, al fine di chiarire il funzionamento interno del modello e le modalità con cui esso elabora il testo in input.
3. **Metodo di ricerca**. In questo capitolo vengono illustrati i dettagli operativi e le scelte metodologiche adottate nel corso del lavoro. In particolare, vengono descritte le fasi di preparazione, pulizia e suddivisione del dataset, evidenziando le strategie utilizzate per garantire coerenza e bilanciamento dei dati. Viene successivamente approfondita l'intera procedura di addestramento del modello. Particolare attenzione è dedicata alla configurazione dei parametri di training, motivandone le scelte.
4. **Risultati**. In questo capitolo vengono presentati e analizzati i risultati ottenuti dal modello sviluppato. In particolare, si esaminano le performance registrate durante la fase di addestramento e validazione tramite le metriche di valutazione adottate, valutando, quindi, la capacità del modello di stimare correttamente il sentiment rispetto ai valori reali. Inoltre, verranno prodotti dei grafici a supporto dell'analisi, consentendo di facilitare l'interpretazione dei risultati sperimentali.
5. **Conclusioni**, il capitolo finale raccoglie le considerazioni generali sui risultati ottenuti e sul lavoro svolto, analizzandone l'impatto e confrontando il prodotto sviluppato con sistemi che assolvono a funzioni analoghe. Vengono inoltre delineate possibili evoluzioni future, quali l'integrazione di ulteriori strumenti di Intelligenza Artificiale per l'analisi e la classificazione dei documenti allegati alle e-mail, con l'obiettivo di arricchire e rendere più completa la valutazione del sentiment.

CAPITOLO 2

Background

2.1 Dataset

La ricerca o la costruzione di un dataset mirato rappresenta uno degli aspetti più critici nell'operazione di fine-tuning di un modello linguistico, soprattutto quando l'obiettivo è l'adattamento a un dominio specifico, come quello delle comunicazioni e-mail in ambito aziendale o personale. Un dataset accuratamente progettato costituisce infatti la base su cui il modello può apprendere le caratteristiche semantiche, pragmatiche e stilistiche proprie del dominio di interesse.

Nel caso specifico delle e-mail aziendali, non risultano disponibili dataset pubblici contenenti messaggi autentici etichettati con un valore numerico di sentiment. Tale assenza è riconducibile sia a problematiche legate alla privacy e alla riservatezza dei dati, sia alla complessità del processo di annotazione. Difatti, anche qualora un dataset di questo tipo fosse reperibile, sarebbe comunque necessario un oneroso lavoro manuale di etichettatura, poiché ogni e-mail dovrebbe essere analizzata e classificata, assegnando un valore di sentiment coerente con il contenuto semantico del testo. Questo processo richiederebbe tempo, competenze specifiche e un significativo impiego di risorse. Una problematica analoga si presenta anche nel caso delle e-mail personali: la natura strettamente privata di queste comunicazioni rende di

fatto impossibile la creazione e la diffusione di dataset pubblici contenenti messaggi autentici, poiché la pubblicazione di tali dati comporterebbe evidenti violazioni della privacy degli utenti. Per superare tale limitazione, si è scelto di utilizzare un dataset alternativo costituito da recensioni della piattaforma Tripadvisor.

Questa scelta è motivata da due vantaggi principali.

- In primo luogo, il dataset presenta etichette già disponibili: ogni recensione è accompagnata da una valutazione espressa dall'utente, che rappresenta implicitamente un indicatore di sentiment. Tale caratteristica consente di evitare un processo manuale di annotazione e di disporre immediatamente di un valore continuo utilizzabile in un contesto di regressione.
- In secondo luogo, il dominio testuale delle recensioni risulta sufficientemente vicino, per struttura e funzione comunicativa, a quello delle e-mail aziendali. Le recensioni sull'erogazione di un servizio, infatti, sono spesso testi mediamente lunghi, articolati e orientati all'espressione di un giudizio soggettivo (positivo, negativo o neutro) dell'esperienza vissuta, analogamente a quanto avviene nelle comunicazioni via e-mail, dove il mittente esprime valutazioni, segnalazioni di criticità o considerazioni su un determinato evento o servizio. Pur non trattandosi di e-mail aziendali autentiche, la similarità strutturale e la presenza di etichette numeriche rendono questo dataset una soluzione metodologicamente coerente e praticabile rispetto agli obiettivi del progetto.

Il dataset utilizzato per l'addestramento del modello è stato reperito dalla piattaforma open source Kaggle, al seguente indirizzo:

<https://www.kaggle.com/datasets/alessandrolobello/italian-tripadvisor>

Il dataset originale era composto da cinque colonne: il testo della recensione, il titolo della recensione, il nickname del recensore, la data di pubblicazione e il rating assegnato.

Per il raggiungimento dello scopo, tuttavia, sono state considerate esclusivamente le colonne relative al testo della recensione e al rating. Le altre informazioni, pur potenzialmente utili per analisi di tipo temporale o comportamentale, non risultavano rilevanti per l'obiettivo specifico dello studio.

2.2 Modello

2.2.1 Origini del modello

L'elaborazione del linguaggio naturale (Natural Language Processing, NLP) ha conosciuto un'evoluzione profonda e progressiva nel corso degli ultimi decenni, passando da approcci simbolici fondati su regole formali a modelli neurali profondi capaci di apprendere automaticamente rappresentazioni semantiche complesse.

Sistemi rule-based I primi sistemi di NLP erano basati su regole simboliche e grammatiche formali costruite manualmente. Linguisti ed esperti definivano strutture sintattiche e semantiche attraverso insiemi di regole deterministiche che descrivevano il funzionamento della lingua, e i sistemi applicavano queste regole per analizzare il testo e produrre un output. Tali sistemi offrivano un'elevata interpretabilità, poiché ogni decisione del sistema poteva essere ricondotta a una regola esplicita. Tuttavia, la complessità e l'ambiguità intrinseca del linguaggio naturale rendevano estremamente difficile coprire tutte le possibili varianti linguistiche. Questi approcci risultavano pertanto rigidi, difficili da aggiornare e scarsamente adattabili a nuovi domini applicativi.

Modelli statistici Successivamente, con l'aumento della disponibilità di dati testuali e della capacità computazionale, si è progressivamente affermato un paradigma statistico. Metodi come Bag-of-Words e TF-IDF hanno permesso di rappresentare i testi come vettori numerici, basati sulla frequenza delle parole nei documenti. Questa trasformazione ha reso possibile l'applicazione di algoritmi di classificazione e analisi del testo, lavorando su una mappa numerica basata sulla frequenza delle parole nel testo. Tuttavia, tali rappresentazioni ignoravano completamente l'ordine delle parole e le relazioni sintattiche, trattando il testo come un insieme non strutturato di termini e valutando ogni termine a se stante. Di conseguenza, veniva persa una parte significativa dell'informazione semantica.

Embedding distribuzionali Un passo avanti rilevante è stato introdotto dagli embedding distribuzionali ovvero rappresentare le parole in spazi vettoriali continui a bassa dimensionalità. In questi modelli, come Word2Vec e GloVe, parole semanticamente simili risultano collocate in regioni vicine dello spazio vettoriale, permettendo di catturare relazioni latenti tra termini. Tale innovazione ha segnato un importante progresso nella modellazione semantica. Tuttavia, queste rappresentazioni erano statiche quindi ogni parola possedeva un unico vettore, indipendentemente dal contesto in cui compariva. Questo limite risultava particolarmente problematico nel caso di parole polisemiche, il cui significato varia in funzione della frase.

Reti neurali ricorrenti Per affrontare il problema della sequenzialità e della dipendenza contestuale, sono state introdotte le reti neurali ricorrenti (RNN). Questi modelli elaboravano il testo parola per parola, mantenendo per ognuna uno stato interno che sintetizzava l'informazione proveniente dagli elementi precedenti della sequenza. In tal modo, diventava possibile modellare il contesto e catturare relazioni temporali tra le parole. Nonostante i miglioramenti, tali architetture presentavano limiti strutturali significativi. La propagazione sequenziale dell'informazione rendeva difficile catturare dipendenze a lungo raggio, mentre durante l'addestramento su sequenze lunghe potevano emergere problemi di vanishing o exploding gradient. Inoltre, la natura sequenziale del calcolo impediva una piena parallelizzazione, impedendo di far lavorare più core/GPU contemporaneamente, rallentando significativamente il training su grandi quantità di dati.

Transformer La necessità di superare queste limitazioni ha portato allo sviluppo di una nuova architettura neurale: il Transformer. Introdotto nel 2017 nel lavoro "Attention Is All You Need", il Transformer si basa su un meccanismo innovativo chiamato self-attention, che consente al modello di analizzare simultaneamente tutti i token di una sequenza. Invece di elaborare il testo in modo strettamente sequenziale, ogni parola può "prestare attenzione" alle altre parole della frase, pesando dinamicamente la loro rilevanza nel determinare la propria rappresentazione.

Questo approccio permette di catturare direttamente le dipendenze a lungo raggio e di costruire rappresentazioni contestuali dinamiche, superando il limite degli embedding statici. Un ulteriore vantaggio fondamentale dell'architettura Transformer è la sua elevata parallelizzabilità. Poiché non è vincolata a un'elaborazione sequenziale, essa può sfruttare in modo efficiente le moderne architetture hardware, come le GPU, rendendo possibile l'addestramento su grandi corpora testuali. Questa caratteristica ha favorito lo sviluppo di modelli pre-addestrati di grandi dimensioni, successivamente adattabili a compiti specifici tramite fine-tuning. L'architettura Transformer è composta da due moduli principali:

- L'Encoder ha il compito di trasformare la sequenza di input in una rappresentazione interna contestualizzata, attraverso una serie di livelli basati su meccanismi di self-attention e reti feed-forward. Ogni token viene quindi arricchito con informazioni provenienti dall'intera sequenza.
- Il Decoder, invece, utilizza tali rappresentazioni per generare una sequenza di output, producendo un token alla volta e tenendo conto sia delle informazioni codificate dall'Encoder sia dei token già generati. Questo schema è particolarmente adatto a compiti di generazione sequenziale, come la traduzione automatica o il text generation.

2.2.2 BERT

Sull'architettura del Transformer si fondano numerosi modelli pre-addestrati che hanno rivoluzionato il NLP moderno. Tra questi, un ruolo centrale è ricoperto da BERT (Bidirectional Encoder Representations from Transformers), introdotto da Google nel 2018. BERT rappresenta un modello basato esclusivamente sulla componente encoder del Transformer e introduce una caratteristica fondamentale: la bidirezionalità completa del contesto. Grazie al Transformer, BERT considera simultaneamente il contesto sia precedente sia successivo rispetto a ciascun token. Questa proprietà consente di ottenere rappresentazioni profondamente contestuali, in cui il significato di una parola è influenzato dall'intera frase.

Il pre-addestramento di BERT avviene tramite due compiti principali: il Masked Language Modeling, in cui alcune parole della frase vengono mascherate e il modello deve predirle, e il Next Sentence Prediction, volto a comprendere la relazione tra coppie di frasi. Grazie a questo processo, il modello apprende rappresentazioni linguistiche generali su grandi corpora testuali, che possono successivamente essere adattate a compiti specifici mediante fine-tuning supervisionato. L'introduzione di BERT ha segnato un punto di svolta nella classificazione testuale e in numerosi altri compiti di NLP, grazie alla sua capacità di comprendere il contesto in modo profondo e dinamico. Nel contesto del presente lavoro, l'utilizzo di un modello basato su architettura Transformer, e in particolare su BERT, consente di sfruttare rappresentazioni contestuali avanzate, migliorando significativamente l'accuratezza e la capacità di generalizzazione rispetto ad altri approcci.

Pertanto, la scelta del modello pre-addestrato di base utilizzato per il fine-tuning ricade su `dbmdz/bert-base-italian-xxl-cased`, una variante di BERT specializzata nella lingua italiana, sviluppata dal gruppo dbmdz (Digital Humanities Group del Munich Center for Machine Learning). Si tratta di un modello Transformer Encoder pre-addestrato su un ampio corpus testuale in italiano (Wikipedia, Open-Subtitles, Gazzetta Ufficiale, giornali, ecc.), che permette di catturare le caratteristiche sintattiche e semantiche della lingua.

2.2.3 Tokenizer

Nelle sezioni precedenti si è parlato di token, poiché i modelli basati su Transformer non operano direttamente sulle parole, ma su queste unità discrete di testo. Un token è una rappresentazione testuale discreta che il modello utilizza come input. Non corrisponde necessariamente a una parola intera: può essere una parola, una sottoparte di parola (subword) o un simbolo speciale.

Il tokenizer è lo strumento associato al modello che trasforma il testo grezzo in sequenze di token. Il tokenizer del modello scelto presenta diverse caratteristiche rilevanti:

- Possiede un vocabolario di circa 32.000 token.
- È definito *cased*, poiché distingue tra maiuscole e minuscole; ad esempio, “Roma” viene tokenizzata in maniera differente da “roma”.
- Utilizza un approccio *WordPiece*, che associa a ciascuna parola presente nel vocabolario un token e suddivide i termini non presenti in esso in sottoparole tokenizzate separatamente, riducendo il problema delle *Out-of-Vocabulary words* (OOV). Ad esempio, la parola “informatizzazione” può essere suddivisa nelle sottoparole [“informat”, “##izzazione”].
- Inserisce alcuni special token utili per la gestione della sequenza e dei compiti di predizione: [CLS], che indica l’inizio della sequenza ed è impiegato in compiti di classificazione o regressione; [SEP], che separa due frasi o indica la fine della sequenza; [PAD], utilizzato per il padding delle sequenze più corte; [MASK], utilizzato durante il pre-training per il compito di Masked Language Modeling.

Il flusso di tokenizzazione segue diversi passaggi principali:

1. **Segmentazione in token:** ogni testo dato al tokenizer viene suddiviso in parole e sottoparole secondo l’algoritmo WordPiece.
2. **Inserimento di special token:** vengono aggiunti token come [CLS] all’inizio della sequenza e [SEP] alla fine o tra frasi.
3. **Generazione di input_ids:** il testo tokenizzato viene convertito in un vettore di interi, in cui ogni token (inclusi gli special token) corrisponde a un identificativo numerico (id) unico e occupa uno slot specifico del vettore. In questo modo, la sequenza di token viene rappresentata come una struttura numerica ordinata, pronta per essere elaborata dal modello Transformer.
4. **Generazione di attention_mask:** viene costruito un vettore binario parallelo alla sequenza degli input_ids, in cui ciascuna posizione corrisponde a uno specifico token. Il valore 1 indica che il token deve essere considerato dal modello durante il calcolo dell’attenzione, mentre il valore 0 identifica i token di padding ([PAD]), che vengono ignorati nel processo computazionale.

Attraverso il processo di tokenizzazione, dunque, il testo grezzo viene trasformato in una rappresentazione numerica uniforme e interpretabile dal modello, permettendo al Transformer di costruire rappresentazioni contestuali per ciascun token e di operare efficacemente nei compiti di analisi del linguaggio naturale, come la predizione del sentiment

2.2.4 Flusso di predizione

Il modello impiegato appartiene alla variante XXL, caratterizzata da un'architettura composta da 24 strati Transformer, una dimensione dello spazio nascosto (hidden size) pari a 1024, 16 teste di self-attention e circa 550 milioni di parametri addestrabili. Questa configurazione conferisce al modello un'elevata capacità rappresentativa, permettendogli di catturare relazioni sintattiche e semantiche complesse anche su sequenze testuali di lunghezza significativa.

Il flusso di predizione sfrutta tali caratteristiche attraverso una serie di passaggi strutturati:

2.2.4.1 Fase di embedding

In questa fase, ciascun valore presente in `input_ids` viene trasformato tramite le embedding matrix di BERT in un vettore denso, la cui dimensione è pari a 1024 nel modello utilizzato. Le embedding matrix sono matrici interne al modello che codificano informazioni distribuzionali sui token, sulle loro posizioni all'interno della sequenza e sul segmento di appartenenza, permettendo al modello di costruire rappresentazioni iniziali ricche e contestualizzate per ogni token. Queste matrici rappresentano parametri del modello che sono stati inizialmente appresi durante la fase di pre-training su grandi corpora testuali. Durante la fase di fine-tuning, tali parametri possono essere ulteriormente aggiornati, consentendo al modello di adattare le rappresentazioni iniziali dei token al compito specifico considerato.

Per ciascun token della sequenza si generano tre vettori distinti della stessa dimensione:

- **Token Embedding:** ottenuto dalla moltiplicazione della Token Embedding Matrix per il valore di `input_ids`, rappresenta il significato semantico base del token.
- **Position Embedding:** ottenuto dalla moltiplicazione della Position Embedding Matrix, codifica la posizione del token nella sequenza (0, 1, 2, ... fino alla lunghezza massima consentita).
- **Segment Embedding:** ottenuto dalla moltiplicazione della Segment Embedding Matrix, distingue tra frasi o segmenti differenti (ad esempio frase A e frase B), aiutando il modello a comprendere strutture più complesse del testo.

I tre vettori vengono poi sommati elemento per elemento per ottenere un unico vettore finale, che rappresenta contemporaneamente il significato del token, la sua posizione e il segmento di appartenenza:

$$E_{\text{finale}} = E_{\text{Token}} + E_{\text{Posizione}} + E_{\text{Segmento}}$$

Questo embedding finale costituisce l'input iniziale per gli strati del Transformer Encoder. In particolare, per ciascun token si dispone di un vettore denso di dimensione $d_{\text{model}} = 1024$ nel modello utilizzato. Se una frase contiene L token, l'intera sequenza viene rappresentata come una matrice $\mathbf{X}$ di dimensione:

$$\mathbf{X} \in \mathbb{R}^{L \times d_{\text{model}}}$$

2.2.4.2 Multi-Head Self-Attention

In questa fase il modello BERT applica il meccanismo di *Multi-Head Self-Attention* alle rappresentazioni vettoriali dei token ottenute nella fase precedente. La self-attention consente a ciascun token della sequenza di valutare l'importanza degli altri token presenti nella frase, permettendo al modello di catturare le relazioni contestuali tra le parole indipendentemente dalla loro distanza nella sequenza.

Il termine *multi-head* indica che il processo di attenzione non viene eseguito una sola volta, ma in parallelo da più componenti chiamate *teste di attenzione*.

Una testa di attenzione è un'unità del meccanismo di self-attention che opera su una proiezione degli embedding dei token in uno spazio vettoriale di dimensione ridotta. In altre parole, ogni testa osserva la sequenza da una prospettiva diversa e apprende specifiche relazioni tra i token. Il modello, dunque, applica self-attention attraverso determinati passi:

1. Proiezioni Q , K e V

Nel meccanismo di *self-attention*, ogni embedding x associato a un token viene trasformato attraverso tre diverse proiezioni lineari, ottenute moltiplicando il vettore di input per tre matrici di parametri (W_Q , W_K , W_V). Tali matrici rappresentano parametri del modello che sono stati inizialmente appresi durante la fase di pre-training di BERT e che, nella fase di fine-tuning, possono essere ulteriormente aggiornati per adattare il comportamento del meccanismo di attenzione al nuovo compito di apprendimento.

$$Q = xW_Q, \quad K = xW_K, \quad V = xW_V$$

dove W_Q , W_K e W_V sono matrici di parametri del modello.

Queste tre trasformazioni producono, per ciascun token, tre nuovi vettori distinti: *Query* (Q), *Key* (K) e *Value* (V), ciascuno con un ruolo specifico nel calcolo dell'attenzione.

La **Query** (Q) può essere interpretata come la “richiesta di informazione” che il token corrente formula nei confronti degli altri token della sequenza. Essa rappresenta quali caratteristiche il token sta cercando nel contesto.

La **Key** (K) rappresenta invece una sorta di “chiave descrittiva” del token: è il vettore con cui gli altri token confronteranno le proprie Query per stabilire il grado di rilevanza reciproca.

Il **Value** (V) contiene l'informazione effettiva che verrà trasmessa agli altri token una volta determinato il livello di attenzione. Se Query e Key stabiliscono quanto un token debba essere considerato rilevante, il Value determina quale contenuto venga effettivamente incorporato.

Le matrici W_Q , W_K e W_V sono, dunque, parametri appresi dal modello e hanno il compito di proiettare l'embedding originale x in tre spazi vettoriali differenti, ciascuno specializzato per il proprio ruolo nel meccanismo di attenzione. Nel caso della *Multi-Head Attention*, come anticipato, tali proiezioni non mantengono la dimensione originale d_{model} . Ogni testa di attenzione lavora infatti in uno spazio vettoriale ridotto, così da poter apprendere rappresentazioni complementari in parallelo. Di conseguenza, da ciascun token non si ottengono vettori Q , K e V di dimensione d_{model} , bensì vettori di dimensione inferiore, indicata con d_k . Questa dimensione è definita come:

$$d_k = \frac{d_{model}}{n_{teste}}$$

In questo modo, ogni testa opera su una porzione dello spazio rappresentativo complessivo, permettendo al modello di catturare simultaneamente diversi tipi di dipendenze linguistiche tra i token, ad esempio relazioni sintattiche, semantiche o di contesto.

2. Prodotto scalare tra Q e K

Una volta ottenuti i vettori *Query* (Q) e *Key* (K) per ciascun token della sequenza, il meccanismo di *self-attention* procede calcolando il grado di rilevanza reciproca tra i token. In particolare, per ogni coppia di token i e j viene calcolato uno *score* di attenzione tramite il prodotto scalare tra la Query del token i e la Key del token j :

$$\text{score}(i, j) = \frac{Q_i \cdot K_j^T}{\sqrt{d_k}}$$

dove d_k rappresenta la dimensionalità dei vettori Q e K per ciascuna testa di attenzione. Il prodotto scalare $Q_i \cdot K_j^T$ misura la similarità tra i due vettori: valori più elevati indicano una maggiore affinità tra la richiesta informativa del token i e le caratteristiche espresse dal token j . In altre parole, questo punteggio quantifica quanto il token i debba “prestare attenzione” al token j durante la fase di aggregazione del contesto.

La divisione per $\sqrt{d_k}$ svolge un ruolo fondamentale nella stabilizzazione numerica del modello.

All'aumentare della dimensionalità dei vettori, il valore atteso del prodotto scalare tende a crescere, rischiando di generare punteggi troppo elevati. Senza questa normalizzazione, la successiva funzione *softmax* produrrebbe distribuzioni estremamente concentrate, riducendo la qualità del gradiente e compromettendo la stabilità del training. Il termine $\sqrt{d_k}$ consente quindi di mantenere i valori in una scala appropriata.

Attraverso questo meccanismo, ogni token viene messo in relazione con tutti gli altri token della sequenza, permettendo al modello di catturare dipendenze sia locali sia a lungo raggio all'interno del testo.

3. Softmax sui pesi di attenzione

I punteggi di attenzione calcolati tra una Query e tutte le Key della sequenza rappresentano valori non normalizzati che indicano il grado di compatibilità tra il token corrente e ciascun altro token. Tuttavia, tali punteggi non sono direttamente interpretabili come pesi, poiché possono assumere valori arbitrari e non rispettano alcun vincolo di somma.

Per ottenere coefficienti interpretabili, viene applicata la funzione *softmax*, che trasforma i punteggi in una distribuzione di probabilità:

$$\alpha_{ij} = \text{softmax}_j(\text{score}(i, j))$$

dove α_{ij} rappresenta il peso di attenzione associato al token j quando si considera il token i .

La funzione softmax svolge due ruoli fondamentali:

- normalizza i valori rendendoli compresi tra 0 e 1;
- impone che la somma dei pesi relativi a una stessa Query sia pari a 1.

In questo modo, i coefficienti α_{ij} possono essere interpretati come il grado di importanza attribuito al token j nella costruzione della rappresentazione del token i .

Un aspetto rilevante è che la softmax enfatizza le differenze tra i punteggi: valori più elevati vengono amplificati esponenzialmente rispetto a quelli più bassi, consentendo al modello di concentrarsi maggiormente sui token più informativi e ridurre l'influenza di quelli meno rilevanti.

4. Combinazione con i vettori Value

Una volta ottenuti i pesi di attenzione α_{ij} tramite la funzione softmax, essi vengono utilizzati per calcolare una combinazione pesata dei vettori V . Per ciascun token i , il nuovo vettore rappresentativo z_i è definito come:

$$z_i = \sum_j \alpha_{ij} V_j$$

Il vettore z_i rappresenta l'output del meccanismo di self-attention per il token i . Esso costituisce una nuova rappresentazione vettoriale del token che incorpora informazioni provenienti da tutti gli altri token della sequenza.

In particolare, il nuovo embedding del token i è ottenuto come una media pesata dei vettori V_j di tutti i token della sequenza, dove i pesi α_{ij} determinano quanto ciascun token contribuisca alla rappresentazione finale.

In termini intuitivi, ogni token costruisce una rappresentazione contestualizzata di sé stesso aggregando le informazioni degli altri token della frase in proporzione alla loro rilevanza, stimata dal meccanismo di attenzione. Il vettore risultante z_i costituisce quindi la rappresentazione contestuale del token prodotta dallo strato di self-attention, che verrà successivamente elaborata dagli ulteriori componenti del blocco Transformer.

Questo processo, come anticipato, non viene eseguito una sola volta, ma ripetuto in parallelo tramite più teste di attenzione (*Multi-Head Attention*). Ogni testa opera in uno spazio vettoriale ridotto di dimensione d_k , così da poter catturare differenti tipologie di relazioni tra i token (ad esempio relazioni sintattiche, semantiche o dipendenze a lungo raggio).

Ciascuna testa produce, per ogni token i , un proprio vettore di output $z_i^{(h)}$, ottenuto applicando il meccanismo di self-attention sui vettori Q , K e V proiettati nello spazio ridotto. I risultati delle diverse teste vengono quindi concatenati e successivamente proiettati tramite una trasformazione lineare nello spazio originale di dimensione d_{model} . La relazione tra le dimensioni è espressa da:

$$n_{heads} \cdot d_k = d_{model}$$

In questo modo il modello combina prospettive multiple sulle relazioni tra i token, mantenendo invariata la dimensione complessiva delle rappresentazioni vettoriali.

2.2.4.3 Feed-Forward Network (FFN)

Dopo il meccanismo di Self-Attention, ciascun token possiede una rappresentazione contestualizzata, ovvero arricchita dalle informazioni provenienti dagli altri token della sequenza. A questo punto, ogni token viene elaborato da una Feed-Forward Network (FFN), una piccola rete neurale completamente connessa applicata in modo identico e indipendente a ciascun embedding. Lo scopo della FFN è quello di trasformare ulteriormente la rappresentazione contestuale prodotta dal meccanismo di self-attention, introducendo maggiore capacità espressiva nel modello. Mentre la self-attention consente ai token di scambiare informazioni tra loro e modellare le relazioni all'interno della sequenza, la Feed-Forward Network opera su ciascun token individualmente, raffinandone la rappresentazione attraverso una serie di trasformazioni lineari e non lineari. In questo modo, la FFN permette al modello di apprendere trasformazioni più complesse delle rappresentazioni vettoriali, migliorando la capacità di estrarre caratteristiche rilevanti dai dati prima che queste vengano propagate agli strati successivi del Transformer.

Formalmente:

$$FFN(x) = GELU(xW_1 + b_1)W_2 + b_2$$

Il funzionamento può essere suddiviso in tre passaggi principali.

1. Proiezione in uno spazio più ampio

Il vettore in input x , di dimensione d_{model} , viene sottoposto a una trasformazione lineare tramite la moltiplicazione per una matrice di pesi W_1 (a cui si aggiunge un termine di bias b_1). Questa operazione proietta il vettore in uno spazio di dimensione superiore, tipicamente pari a $4 \cdot d_{model}$.

Questa espansione temporanea aumenta il numero di componenti del vettore che descrivono il token, permettendo alla rete di rappresentare un numero maggiore di combinazioni tra le caratteristiche presenti nell'embedding. In questo modo il modello dispone di uno spazio vettoriale più ampio in cui poter apprendere trasformazioni più espressive delle informazioni estratte nella fase di self-attention.

2. Introduzione della non linearità tramite GELU

Successivamente viene applicata la funzione di attivazione GELU (*Gaussian Error Linear Unit*). Questa funzione introduce una trasformazione non lineare sugli elementi del vettore ottenuto dal primo layer lineare della Feed-Forward Network.

A differenza della funzione ReLU, che azzerava completamente tutti i valori negativi e mantiene invariati quelli positivi, la GELU applica una trasformazione più graduale e continua. In particolare, l'attivazione di un neurone (ovvero di una singola componente del vettore che rappresenta l'embedding del token) non dipende esclusivamente dal segno del valore in ingresso, ma viene modulata in maniera progressiva: valori piccoli o negativi vengono attenuati, mentre valori positivi e di maggiore intensità vengono mantenuti con un peso più elevato. Matematicamente, la funzione GELU può essere espressa come:

$$\text{GELU}(x) = x \Phi(x)$$

dove $\Phi(x)$ rappresenta la funzione di distribuzione cumulativa della distribuzione normale standard. Tale funzione restituisce un valore compreso tra 0 e 1 che agisce come coefficiente di modulazione dell'input x . Di conseguenza, l'output della GELU è ottenuto ridimensionando il valore di ingresso in modo continuo: valori molto negativi vengono fortemente attenuati, valori prossimi allo zero vengono ridotti, mentre valori positivi più elevati vengono mantenuti quasi inalterati.

In termini intuitivi, la GELU non introduce una soglia netta come la ReLU, ma applica una modulazione continua dei valori del vettore, permettendo al modello di preservare in parte anche informazioni deboli o intermedie presenti nei dati. L'introduzione della non linearità è fondamentale: senza di essa, l'intero blocco della Feed-Forward Network sarebbe equivalente a una semplice trasformazione lineare, limitando la capacità del modello di apprendere relazioni complesse tra le variabili. L'utilizzo della GELU, adottata nei modelli BERT e in molte architetture Transformer, consente inoltre di ottenere una transizione più fluida tra valori attivati e non attivati, contribuendo a migliorare la capacità espressiva del modello e la stabilità del processo di addestramento.

3. Proiezione di ritorno

Il risultato viene infine moltiplicato per la matrice W_2 (a cui si aggiunge un termine di bias b_2), che riporta il vettore alla dimensione originale d_{model} . In questo modo, ogni token ottiene una rappresentazione arricchita ma compatibile dimensionalmente con gli strati successivi del modello.

I termini di bias b_1 e b_2 vengono sommati al risultato delle rispettive trasformazioni lineari e consentono al modello di traslare le funzioni di trasformazione apprese, aumentando la flessibilità del modello e permettendo di adattare meglio le rappresentazioni interne dei token. Tali parametri, insieme alle matrici di pesi W_1 e W_2 , sono stati inizialmente appresi durante il pre-training del modello BERT e, nella fase di fine-tuning, possono essere ulteriormente aggiornati tramite il processo di ottimizzazione per adattare le rappresentazioni interne al nuovo compito di apprendimento.

2.2.4.4 Residual Connections e Layer Normalization

Per garantire stabilità numerica e facilitare l'addestramento di reti neurali profonde come i Transformer, i blocchi principali dell'architettura sono racchiusi in una struttura composta da *Residual Connection* e *Layer Normalization*. Questi due meccanismi permettono di preservare l'informazione originale lungo i vari strati della rete, favorire la propagazione del gradiente durante la predizione e mantenere stabili le rappresentazioni vettoriali apprese dal modello. Sia il blocco di Self-Attention sia quello di Feed-Forward Network (FFN) sono quindi racchiusi in questa struttura, formalmente descritta come:

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

dove x rappresenta l'input del blocco e $\text{Sublayer}(x)$ indica la trasformazione applicata dal sotto-blocco considerato (Self-Attention o FFN).

Residual Connection La connessione residua consiste nella somma tra l'input originale x e l'output prodotto dal sotto-blocco. In questo modo, l'informazione iniziale viene preservata e propagata direttamente agli strati successivi, anche nel caso in cui il sublayer apprenda trasformazioni poco significative.

Questo meccanismo contribuisce inoltre a rendere più stabile il processo di addestramento delle reti profonde. Nei modelli caratterizzati da molti strati, infatti, le rappresentazioni possono degradarsi progressivamente man mano che attraversano i vari livelli della rete. Le residual connections permettono invece di mantenere un collegamento diretto con l'input originale, facilitando la propagazione dell'informazione e consentendo un apprendimento più efficace in architetture profonde come i Transformer.

Layer Normalization Dopo la somma residua, viene applicata la *Layer Normalization*, una tecnica di normalizzazione utilizzata per rendere più stabili le rappresentazioni interne del modello durante l'addestramento.

In particolare, la normalizzazione viene applicata alla rappresentazione vettoriale del singolo token. Per ciascuna rappresentazione vengono calcolate la media e la deviazione standard dei suoi valori. Successivamente, ogni componente della rappresentazione viene normalizzata sottraendo la media e dividendo per la deviazione standard. In questo modo, i valori risultano distribuiti in maniera più uniforme. Formalmente, la Layer Normalization può essere espressa come:

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sigma} \gamma + \beta$$

dove μ e σ rappresentano rispettivamente la media e la deviazione standard dei valori della rappresentazione x , mentre γ e β sono parametri addestrabili della Layer Normalization. Nel caso dei modelli BERT pre-addestrati, tali parametri sono già stati appresi durante il pre-training del modello. Durante la fase di fine-tuning, essi possono essere ulteriormente aggiornati insieme agli altri pesi della rete tramite il processo di ottimizzazione.

2.2.4.5 Regression Head

La *Regression Head* rappresenta l'ultimo stadio del flusso di predizione del modello. Dopo che i blocchi di Multi-Head Self-Attention, Feed-Forward Network (FFN) e le relative strutture di Residual Connection e Layer Normalization vengono iterati per N volte, dove N corrisponde al numero di strati del modello utilizzato (24 in questo caso), ciascun embedding di token contiene informazioni sia sul token stesso sia sull'intero contesto della sequenza.

Grazie al meccanismo di self-attention, ogni token ha infatti avuto la possibilità di "osservare" gli altri token e integrare progressivamente le loro informazioni nella propria rappresentazione.

In particolare, il token speciale [CLS], aggiunto automaticamente all'inizio della sequenza dal tokenizer, è progettato per fungere da rappresentazione globale della frase. Durante i vari strati di attenzione, il token [CLS] interagisce con tutti gli altri token e incorpora le loro informazioni nel proprio embedding finale.

L'ultimo passo del processo di predizione utilizza proprio l'embedding finale del token [CLS] come input alla Regression Head. Quest'ultima è costituita da un layer lineare (*fully connected*) che produce un singolo valore continuo, rappresentante lo score di sentiment stimato per l'intera frase.

Formalmente:

$$y_{pred} = W \cdot h_{[CLS]} + b$$

dove:

- $h_{[CLS]}$ è l'embedding finale del token [CLS] (di dimensione d_{model}), che rappresenta una sintesi delle informazioni dell'intera sequenza di input;
- W è il vettore dei pesi della *regression head* (di dimensione $1 \times d_{model}$). Ogni componente di W determina il contributo della corrispondente dimensione dell'embedding $h_{[CLS]}$ alla predizione finale. Durante la fase di addestramento, questi pesi vengono aggiornati, permettendo al modello di apprendere quali caratteristiche della rappresentazione contestuale del testo risultino maggiormente informative per la stima del sentimento;

- b è un termine di bias scalare ($b \in \mathbb{R}$) che viene sommato al risultato del prodotto tra W e $h_{[\text{CLS}]}$. Il bias consente di traslare la funzione di predizione del modello, permettendo una maggiore flessibilità nell'adattamento ai dati. Analogamente ai pesi W , anche il bias viene aggiornato durante la fase di addestramento.
- y_{pred} è il valore continuo prodotto dal modello, interpretato come score di regressione del sentimento.

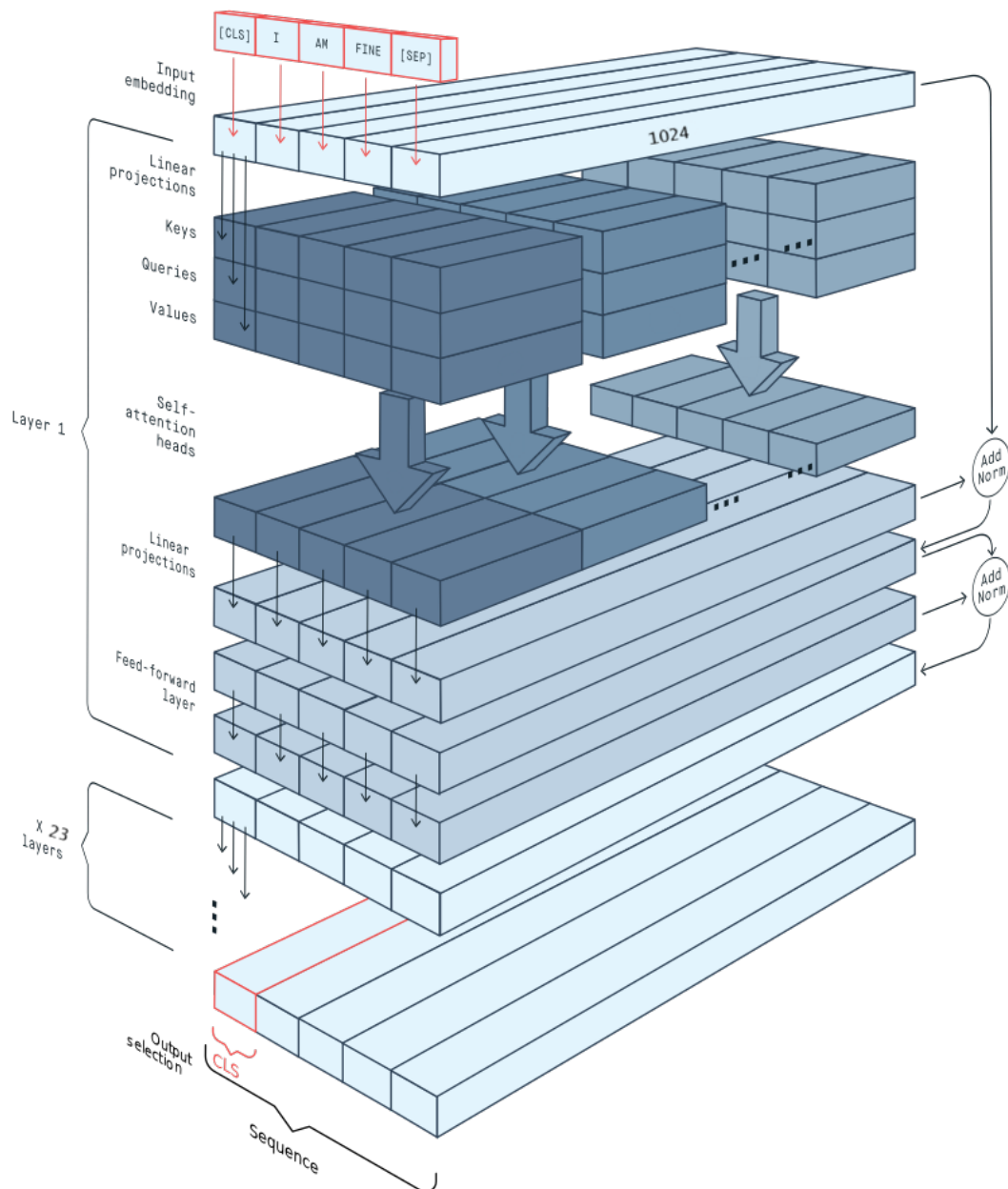


Figura 2.1: Illustrazione grafica del flusso di predizione di BERT

3.1 Sviluppo applicativo

L'intero processo di preparazione dei dati, addestramento e valutazione del modello è stato sviluppato utilizzando il linguaggio di programmazione Python all'interno dell'ambiente Jupyter Notebook. Quest'ultimo è stato scelto per la sua capacità di integrare codice eseguibile, testo descrittivo e visualizzazioni grafiche nello stesso ambiente interattivo, consentendo di documentare e monitorare tutte le fasi del processo di analisi. La scelta di Python è motivata dal suo ruolo centrale nel panorama della data science e del machine learning. Il linguaggio mette infatti a disposizione un ecosistema molto ricco di librerie dedicate alla manipolazione dei dati, all'elaborazione del linguaggio naturale, allo sviluppo di modelli di apprendimento automatico e alla visualizzazione dei risultati.

Grazie a questi strumenti, è possibile costruire pipeline complete per l'analisi dei dati e l'addestramento di modelli di deep learning, in particolare quelli basati su architetture Transformer, oltre a disporre di strumenti per la valutazione delle prestazioni tramite metriche statistiche e rappresentazioni grafiche dei risultati.

3.2 Raffinamento del dataset

Un aspetto rilevante nella scelta del dataset riguarda il possibile fenomeno di *domain shift*, ossia la discrepanza tra il dominio dei dati utilizzati per il fine-tuning (recensioni online) e quello applicativo finale (e-mail aziendali e personali).

Sebbene entrambe le tipologie testuali condividano la natura discorsiva e l'espressione di un giudizio soggettivo, esse differiscono per stile comunicativo, lessico e finalità pragmatica. Le recensioni tendono infatti a esprimere valutazioni esplicite relative a un servizio o prodotto, mentre le e-mail, in particolare quelle aziendali, possono veicolare criticità in maniera più implicita, formale e contestualizzata all'interno di dinamiche organizzative. Questa divergenza può influire sulle prestazioni del modello, in quanto la distribuzione statistica dei dati di addestramento potrebbe non coincidere pienamente con quella dei dati reali in fase di utilizzo. Tuttavia, l'impiego di un modello linguistico pre-addestrato consente di partire da rappresentazioni semantiche generali e robuste, riducendo l'impatto della differenza di dominio e favorendo la capacità di generalizzazione.

3.2.1 Pre-processing dei dati

Il dataset di recensioni presenta alcune criticità tipiche dei testi non strutturati, quali la presenza di emoji, link esterni, righe vuote o elementi testuali superflui.

Tali componenti non contribuiscono in modo significativo alla comprensione semantica del contenuto e possono introdurre rumore durante la fase di addestramento del modello, influenzandone negativamente la convergenza e la capacità di apprendimento.

Per questo motivo, tutte le recensioni sono state sottoposte a una fase sistematica di pre-processing che ha previsto: rimozione di URL e collegamenti esterni; eliminazione di emoji e simboli non testuali; filtraggio di righe vuote o prive di contenuto informativo. Questa operazione di pulizia ha consentito di ottenere un corpus più coerente e omogeneo, migliorando la qualità dell'input fornito al modello e riducendo la presenza di elementi non rilevanti ai fini dell'analisi del sentiment.

```

1 df = pd.read_csv("dataItalian.csv")
2
3 def clean_text(text):
4     if pd.isnull(text):
5         return ""
6     text = str(text)
7     text = re.sub(r"http\S+|www\.\S+", "", text)
8     text = emoji.replace_emoji(text, replace='')
9     text = "\n".join([line for line in text.splitlines()
10         if line.strip() != ""])
11     return text.strip()
12
13 df['review_text'] = df['review_text'].apply(clean_text)

```

3.2.2 Mapping delle etichette in ottica regressiva

Il dataset originale fornisce come etichetta un valore intero compreso tra 1 e 5 stelle, rappresentativo del giudizio espresso dall'utente. Tuttavia, il modello sviluppato affronta il problema come task di regressione, producendo un output continuo nell'intervallo 0,1.

Per rendere le etichette compatibili con questo approccio, è stato applicato un mapping lineare delle valutazioni:

Valore iniziale	Soglia continua
1	0.00
2	0.25
3	0.50
4	0.75
5	1.00

Tabella 3.1: Tabella di conversione dell'etichetta iniziale del dataset in soglie continue

Questa trasformazione preserva la natura ordinale delle valutazioni a stelle, mantenendone le relazioni di distanza relativa, e consente al modello di apprendere una rappresentazione continua del sentiment. L'approccio regressivo offre inoltre maggiore flessibilità in fase di predizione, permettendo di ottenere punteggi intermedi e quindi una graduazione più fine del livello di criticità.

```
1 label_map = {1.0: 0.0, 2.0: 0.25, 3.0: 0.50, 4.0: 0.75, 5.0: 1.0}
2 df['labels'] = df['review_stars'].map(label_map)
```

3.2.3 Bilanciamento del dataset

L'analisi preliminare del dataset ha evidenziato un marcato bilanciamento nella distribuzione delle etichette.

Valore mappato	Numero recensioni
1.00	108.016
0.75	32.119
0.50	18.558
0.25	4.763
0.00	3.512

Tabella 3.2: Censimento del dataset preliminare

In particolare, la classe corrispondente al valore 1.00 (5 stelle) risulta nettamente predominante rispetto alle altre, mentre le classi intermedie, soprattutto quella associata a 0.25 (2 stelle), risultano fortemente sottorappresentate. Uno squilibrio di tale entità avrebbe potuto compromettere il processo di addestramento, inducendo il modello a privilegiare le predizioni verso le classi maggiormente rappresentate, con conseguente riduzione della capacità discriminativa nei confronti dei casi minoritari. Sono state pertanto valutate due possibili strategie di bilanciamento:

- Oversampling delle classi minoritarie, mediante generazione artificiale di nuovi esempi (ad esempio tramite tecniche di data augmentation basate sulla scomposizione e ricombinazione dei periodi). Tuttavia, l'elevato divario numerico tra la classe maggioritaria e quella minoritaria avrebbe comportato la creazione di un corpus ridondante, con numerose frasi ripetute, aumentando il rischio di overfitting e riducendo la capacità di generalizzazione del modello.
- Downsampling delle classi maggioritarie, ossia la riduzione del numero di esempi appartenenti alle classi più frequenti fino ad allinearle alla numerosità della classe meno rappresentata.

Si è optato, in conclusione, per la seconda strategia, in quanto metodologicamente più solida nel contesto specifico. Tutte le classi sono state quindi portate a 3.512 recensioni ciascuna, ottenendo un dataset finale bilanciato composto da 17.560 istanze complessive.

Questa scelta consente di garantire un apprendimento più equo tra le diverse fasce di sentiment, migliorando la stabilità del training e la capacità del modello di distinguere in modo efficace tra differenti livelli di criticità.

```
1 class_counts = df['labels'].value_counts()
2 min_count = class_counts.min()
3
4 df = pd.concat([resample(df[df['labels'] == label], replace=False,
5     n_samples=min_count, random_state=42)
6     for label in class_counts.index
7 ])
8
9 df = df.sample(frac=1, random_state=42).reset_index(drop=True)
```

3.3 Suddivisione del dataset

Per far fronte alla misura della robustezza del modello, ossia alla sua capacità di mantenere performance stabili anche su dati non osservati (come definito nella Sottosezione 1.3.1) si è adottata una suddivisione strutturata del dataset. In particolare, i dati sono stati ripartiti in *training set*, *validation set* e *test set*, secondo una procedura standard nel machine learning nota come *hold-out validation*, o più precisamente *three-way split*.

Tale metodologia consiste nel suddividere l'insieme complessivo dei dati disponibili in tre sottoinsiemi distinti, ciascuno con una funzione specifica nel processo di addestramento, ottimizzazione e valutazione del modello. In questo caso è stata adottata una ripartizione pari al 70% per il training set, 15% per il validation set e 15% per il test set.

- Il *training set* (70%) è utilizzato per aggiornare i pesi del modello durante la fase di addestramento. In questa fase il modello apprende le relazioni tra l'input testuale e il valore di sentiment associato, attraverso l'ottimizzazione dei parametri interni.
- Il *validation set* (15%) viene impiegato durante il training per monitorare le prestazioni del modello su dati non direttamente utilizzati per l'apprendimento. Esso consente di valutare la qualità dell'apprendimento, ottimizzare gli iperparametri e individuare eventuali fenomeni di sovradattamento (*overfitting*). Il validation set non contribuisce all'aggiornamento dei pesi, ma guida le scelte progettuali relative alla configurazione del modello.
- Il *test set* (15%) è invece riservato a una valutazione finale, condotta esclusivamente al termine della fase di sviluppo. Esso non è mai visibile al modello né durante il training né durante il processo di tuning degli iperparametri. L'utilizzo del test set consente di ottenere una stima imparziale della capacità di generalizzazione del modello su dati completamente nuovi.

Questa separazione evita fenomeni di *data leakage*, poiché gli elementi del dataset utilizzati durante la fase di training non vengono riproposti nelle fasi di validazione e di testing. In questo modo si evita di ottenere prestazioni artificialmente elevate dovute al fatto che il modello abbia già "visto" parte dei dati durante l'addestramento, e si rende possibile una stima più realistica delle prestazioni del sistema in scenari applicativi reali. La proporzione 70/15/15 è stata scelta per mantenere un adeguato bilanciamento tra la quantità di dati disponibili per l'addestramento del modello e l'affidabilità delle fasi di validazione e valutazione finale.

La suddivisione, inoltre, è stata effettuata in modo casuale ma riproducibile, mediante l'utilizzo di un *seed* fisso, al fine di garantire la replicabilità degli esperimenti. Inoltre, è stata preservata la distribuzione delle classi nei tre sottoinsiemi attraverso uno *split stratificato*, così da mantenere inalterato il bilanciamento delle categorie di sentiment all'interno di ciascun set.

```
1 dataset = Dataset.from_pandas(df)
2 dataset = dataset.train_test_split(test_size=0.3, seed=42)
3 splitValidation = dataset['test'].train_test_split(test_size=0.5, seed=42)
4
5 dataset = DatasetDict({
6     'train': dataset['train'],
7     'test': splitValidation['train'],
8     'validation': splitValidation['test']
9 })
```

3.4 Caricamento modello e tokenizzazione

Dopo la fase di pre-processing, bilanciamento e suddivisione del dataset, si procede con il caricamento del modello pre-addestrato e con la tokenizzazione dell'intero dataset ai fini del fine-tuning.

```
1 MODEL = "dbmdz/bert-base-italian-xxl-cased"
2 tokenizer = AutoTokenizer.from_pretrained(MODEL)
3
4 def tokenize(batch):
5     return tokenizer(batch['review_text'], padding='max_length',
6                       truncation=True, max_length=512)
7
8 dataset = dataset.map(tokenize, batched=True)
9
10 model = AutoModelForSequenceClassification.from_pretrained(MODEL,
11                   num_labels=1, problem_type="regression")
```

Viene inizializzato il tokenizer associato al modello pre-addestrato e applicata la funzione di tokenizzazione all'intero dataset in particolare alla colonna contenente le recensioni degli utenti. Tale operazione restituisce una nuova struttura dati che, oltre alle colonne originali (testo della recensione ed etichetta), include le colonne generate automaticamente dal tokenizer, tra cui `input_ids` e `attention_mask` (come illustrato nella Sezione 2.2.3), convertendo ciascun testo in una sequenza numerica compatibile con l'architettura Transformer.

Durante questa fase viene inoltre specificata una lunghezza massima della sequenza pari a 512 token, coerentemente con il limite architetturale del modello. Qualora una sequenza risulti più corta, il tokenizer aggiunge token speciali di padding [PAD] e assegna valore 0 alle corrispondenti posizioni del vettore `attention_mask`, fino al raggiungimento della lunghezza prefissata. Al contrario, nel caso in cui una sequenza superi il limite massimo, i token eccedenti vengono troncati, garantendo così uniformità dimensionale tra tutti gli input.

Successivamente viene caricato il modello specificando `num_labels = 1` e `problem_type = "regression"`. La scelta di impostare `num_labels` a 1 deriva dalla natura del problema affrontato: il modello non deve predire una classe discreta (ad esempio positivo/negativo), bensì un unico valore numerico continuo rappresentante il livello di sentiment del testo. Di conseguenza, l'output finale del modello consiste in un singolo neurone che restituisce un valore reale.

L'impostazione `problem_type = "regression"` consente al framework di configurare automaticamente la funzione di perdita più appropriata: la Mean Squared Error (MSE). Questo permette al modello di ottimizzare direttamente l'errore tra il valore predetto e il valore reale continuo associato a ciascun elemento del dataset.

3.5 Addestramento

La fase di addestramento del modello `dbmdz/bert-base-italian-xxl-cased` per la predizione del sentiment tramite regressione è stata realizzata utilizzando la classe `Trainer` della libreria Hugging Face Transformers, che automatizza gran parte del processo di addestramento.

3.5.1 Argomenti del trainer

L'oggetto `Trainer` integra in un'unica interfaccia tutti gli elementi necessari alla gestione del processo di training, ricevendo in input i seguenti componenti fondamentali:

```
1 trainer = Trainer(  
2     model=model,  
3     args=training_args,  
4     train_dataset=dataset['train'],  
5     eval_dataset=dataset['test'],  
6     processing_class=tokenizer,  
7     data_collator=collate_fn,  
8     compute_metrics=compute_metrics,  
9     callbacks=[EarlyStoppingCallback(early_stopping_patience=2)]  
10 )
```

- **model**: il modello neurale che verrà addestrato e che si occuperà dell'elaborazione dei dati in ingresso;
- **args**: l'insieme degli iperparametri di addestramento (`TrainingArguments`), che definiscono configurazioni relative all'addestramento del modello.

- **train_dataset** ed **eval_dataset**: rispettivamente il dataset utilizzato per l'addestramento e quello impiegato per la validazione del modello durante il training;
- **tokenizer** (`processing_class`): il componente responsabile della trasformazione del testo grezzo attraverso operazioni di tokenizzazione, padding e troncamento (come illustrato nella sezione 2.2.3);
- **data_collator**: funzione responsabile della costruzione dei batch, ossia dell'aggregazione di più elementi del dataset in un unico raggruppamento. Essa, inoltre, converte i dati in tensori, ovvero strutture numeriche multidimensionali utilizzate nei framework di deep learning per rappresentare matrici e array su cui il modello può eseguire operazioni matematiche in modo efficiente. In tal modo viene garantita una struttura coerente, numericamente compatibile e pronta per l'elaborazione da parte del modello;
- **compute_metrics**: la funzione che calcola le metriche di valutazione al termine di ciascun ciclo di addestramento (ad esempio MSE, MAE e R^2), consentendo di monitorare l'andamento delle prestazioni;
- **Callback di Early Stopping**: il meccanismo che interrompe automaticamente l'addestramento nel caso in cui la metrica monitorata non mostri miglioramenti per un numero prefissato di epoche consecutive (nel caso specifico, due epoche consecutive), prevenendo fenomeni di overfitting e ottimizzando l'utilizzo delle risorse computazionali.

3.5.2 Configurazione degli iperparametri

Gli iperparametri non vengono appresi dal modello durante il training, ma sono configurati *a priori* per controllare la dinamica dell'ottimizzazione e la stabilità del processo di apprendimento. Nell'ambito della libreria Hugging Face, essi vengono raggruppati e gestiti tramite l'oggetto `TrainingArguments`, che consente di definire in modo strutturato tutte le configurazioni relative all'addestramento del modello. Gli iperparametri impostati sono:

```
1 training_args = TrainingArguments(  
2     output_dir="./sentiment_regression",  
3     eval_strategy="epoch",  
4     save_strategy="epoch",  
5     per_device_train_batch_size=4,  
6     per_device_eval_batch_size=4,  
7     logging_strategy="epoch",  
8     load_best_model_at_end=True,  
9     learning_rate=2e-5,  
10    num_train_epochs=10,  
11    weight_decay=0.01,  
12    logging_dir='./logs',  
13    fp16=True,  
14    gradient_accumulation_steps=8,  
15 )
```

- `output_dir = "./sentiment_regression"`

Determina la cartella in cui vengono salvati i checkpoint del modello e i file di configurazione. Alla fine di ogni ciclo di addestramento il Trainer crea una sottocartella in questa directory.

- `eval_strategy = "epoch"`

Stabilisce quando effettuare la valutazione sul dataset di validazione. In questo caso la valutazione avviene al termine di ogni epoca, ovvero dopo un passaggio completo sull'intero dataset di addestramento.

- `save_strategy = "epoch"`

Determina quando salvare i checkpoint del modello. Un checkpoint rappresenta una copia salvata dello stato del modello durante l'addestramento, contenente i pesi appresi e altre informazioni necessarie per riprendere il training o recuperare una specifica versione del modello ottenuta in una determinata fase dell'addestramento. Il salvataggio avviene alla fine di ogni epoca all'interno della directory specificata in `output_dir`.

- `fp16 = True`

Abilita il *mixed precision training* (16-bit floating point). Riduce l'uso di memoria GPU e velocizza l'addestramento mantenendo un'elevata precisione. Parte dei calcoli viene eseguita in FP16, mentre le operazioni sensibili (accumulo

dei gradienti e aggiornamento dei pesi) restano in FP32 per garantire stabilità numerica.

- `per_device_train_batch_size = 4`
Numero di istanze incluse in ogni batch durante l'addestramento.
- `per_device_eval_batch_size = 4`
Numero di istanze per batch durante la valutazione. Influisce sulla velocità di evaluation ma non sull'aggiornamento dei pesi.
- `logging_strategy = "epoch"`
Frequenza di registrazione dei log di addestramento. In questo caso il logging avviene alla fine di ogni epoca.
- `logging_dir = "./logs"`
Specifica la cartella in cui vengono salvati i log di addestramento.
- `load_best_model_at_end = True`
Al termine dell'addestramento viene caricato automaticamente il modello con le migliori prestazioni secondo la metrica di valutazione scelta (MSE).
- `num_train_epochs = 10`
Numero massimo di epoche, ovvero il numero di passaggi completi sul dataset di addestramento (salvo interruzione anticipata tramite *Early Stopping*).
- `learning_rate =  $2 \times 10^{-5}$`
Definisce il tasso di apprendimento iniziale dell'ottimizzatore, controlla l'entità delle modifiche apportate ai parametri del modello ad ogni aggiornamento. Un valore adeguato è fondamentale per garantire stabilità durante il fine-tuning: valori troppo elevati potrebbero compromettere le rappresentazioni linguistiche apprese in fase di pre-addestramento, mentre valori troppo bassi renderebbero il processo di convergenza eccessivamente lento. Tipicamente si adottano valori compresi tra 10^{-5} e 5×10^{-5} . Il learning rate viene progressivamente ridotto tramite uno scheduler, consentendo una fase iniziale più dinamica e una fase finale più stabile e orientata alla convergenza. (L'utilizzo viene approfondito nella sezione 3.5.3.6)

- `gradient_accumulation_steps = 8`

Numero di batch su cui vengono accumulati i gradienti prima di effettuare un aggiornamento dei parametri. Consente di simulare un batch effettivo più grande senza saturare la memoria GPU. (L'utilizzo viene approfondito nella sezione 3.5.3.5)

- `weight_decay = 0.01`

Il weight decay agisce introducendo una penalizzazione che tende a ridurre progressivamente la magnitudine dei pesi durante l'addestramento. In pratica, ad ogni aggiornamento, oltre alla correzione dovuta al gradiente della loss, viene applicata una leggera "spinta" che impedisce ai parametri di assumere valori troppo elevati. Questo meccanismo evita che il modello si specializzi eccessivamente sui dati di training, limitando la tendenza a memorizzare dettagli specifici.

Mantenendo i pesi su valori più contenuti, il modello risulta generalmente più stabile e meno sensibile alle variazioni dei dati, migliorando così la capacità di generalizzazione su esempi non visti. Tale accorgimento assume particolare rilevanza nel fine-tuning di modelli di grandi dimensioni, dove l'elevato numero di parametri può favorire fenomeni di overfitting se non adeguatamente controllato. (L'utilizzo viene approfondito nella sezione 3.5.3.6)

3.5.3 Funzionamento del trainer

Una volta avviato l'addestramento, il flusso operativo può essere suddiviso nelle seguenti fasi principali.

3.5.3.1 Preparazione dei batch

Il Trainer preleva i dati dal `train_dataset` e utilizza il `data_collator` per costruire i batch di addestramento. Prima di questa fase, i testi sono stati sottoposti a tokenizzazione.

Il `data_collator` aggrega quindi più elementi tokenizzati in un unico batch e li converte in tensori. I batch costruiti vengono automaticamente trasferiti sul dispositivo di calcolo disponibile (GPU, se presente, altrimenti CPU), ottimizzando le operazioni computazionali e rendendo i dati pronti per il forward pass all'interno del modello.

3.5.3.2 Forward Pass

Per ogni batch, i token delle diverse sequenze vengono forniti in ingresso al modello BERT. Attraverso il meccanismo di self-attention e i layer Transformer, il modello produce una rappresentazione contestuale per ciascun token della sequenza. Nel task di regressione considerato, per ogni sequenza del batch viene utilizzata la rappresentazione associata al token speciale `[CLS]`, che funge da vettore riassuntivo dell'intero contenuto testuale. Tale rappresentazione viene fornita in ingresso a un layer lineare finale (regression head), il quale restituisce un valore numerico continuo che rappresenta la predizione del sentimento (come illustrato nella sezione 2.2.4).

3.5.3.3 Calcolo della Loss

Poiché il batch contiene più sequenze, il modello produce una predizione per ciascun elemento del batch. La funzione di perdita viene quindi calcolata confrontando ogni predizione con la corrispondente etichetta reale e successivamente mediata sull'intero batch. Avendo impostato il problema come regressione nel caricamento del modello su cui effettuare fine-tuning, il framework configura automaticamente come funzione di perdita la *Mean Squared Error* (MSE).

3.5.3.4 Backpropagation

Una volta calcolata la loss, viene applicato l'algoritmo di backpropagation, che consente di determinare il gradiente della funzione di perdita rispetto a ciascun parametro del modello:

$$\frac{\partial Loss}{\partial W}$$

Il termine $\frac{\partial \text{Loss}}{\partial W}$ rappresenta la derivata parziale della loss rispetto al parametro W e misura quanto la funzione di perdita varia in seguito a una variazione infinitesima di quel singolo parametro, mantenendo costanti tutti gli altri. In altre parole, esso esprime la sensibilità della loss rispetto a W , indicando quanto l'errore aumenti o diminuisca al variare di tale peso. Il segno del gradiente indica la direzione della variazione: un valore positivo segnala che un aumento di W comporta un incremento della loss, mentre un valore negativo indica che un aumento del parametro ne determinerebbe una diminuzione. Il valore assoluto del gradiente, invece, riflette l'intensità dell'influenza di quel parametro sull'errore complessivo.

Poiché BERT contiene circa 550 milioni di parametri addestrabili, il gradiente viene calcolato per ciascuno di essi, fornendo un'indicazione puntuale su quanto e in quale direzione ogni parametro debba essere modificato per ridurre l'errore globale. L'aggiornamento dei parametri avviene nella direzione opposta al gradiente, secondo il principio della discesa del gradiente, così da minimizzare progressivamente la funzione di perdita nel corso delle iterazioni di training.

3.5.3.5 Gradient Accumulation

Per ottimizzare l'utilizzo della memoria, il Trainer implementa la tecnica di *gradient accumulation*. I gradienti vengono accumulati su più batch prima di effettuare l'aggiornamento dei pesi dei parametri, simulando di fatto un batch size maggiore senza incrementare il consumo di memoria.

L'utilizzo di batch di dimensioni maggiori produce stime del gradiente meno soggette a varianza, poiché tiene conto di un numero più ampio di esempi per ogni aggiornamento. Ciò riduce le oscillazioni durante l'ottimizzazione e favorisce una convergenza più stabile. Tuttavia, nel fine-tuning di modelli di grandi dimensioni come BERT, la memoria disponibile può non essere sufficiente per gestire batch molto estesi. La gradient accumulation permette quindi di ottenere i benefici di un batch size maggiore senza superare i limiti hardware, mantenendo al contempo una dinamica di apprendimento più regolare.

3.5.3.6 Ottimizzazione con AdamW

L'aggiornamento dei parametri viene effettuato mediante l'ottimizzatore AdamW, variante dell'algoritmo Adam che introduce una gestione più efficace della regolarizzazione tramite il meccanismo di *decoupled weight decay*.

Nel caso generale, l'aggiornamento dei parametri può essere espresso come:

$$W_{\text{nuovo}} = W_{\text{vecchio}} - \eta \frac{\partial \text{Loss}}{\partial W}$$

dove η rappresenta il learning rate e $\frac{\partial \text{Loss}}{\partial W}$ il gradiente della funzione di perdita rispetto ai parametri.

In presenza di weight decay, la regola di aggiornamento include anche un termine di penalizzazione proporzionale al valore corrente dei parametri:

$$W_{\text{nuovo}} = W_{\text{vecchio}} - \eta \left(\frac{\partial \text{Loss}}{\partial W} + \lambda W_{\text{vecchio}} \right)$$

dove λ rappresenta il coefficiente di regolarizzazione.

3.5.3.7 Logging, valutazione ed Early Stopping

Al termine di un'epoca di addestramento, ovvero dopo che il `Trainer` ha processato sequenzialmente tutti i batch del `train_dataset` attraverso i passaggi precedentemente illustrati, in funzione delle impostazioni di `logging_strategy`, `evaluation_strategy` e `save_strategy`, esegue una serie di operazioni finalizzate al monitoraggio dell'addestramento e alla valutazione delle prestazioni del modello. In particolare:

- calcola e registra la training loss media dell'epoca, ottenuta come media delle loss calcolate sui singoli batch del training set durante il processo di addestramento;
- calcola le metriche definite nella funzione `compute_metrics` (MSE, MAE e coefficiente di determinazione R^2) confrontando le predizioni del modello con le etichette reali del validation set (`eval_dataset`);
- registra nei log le informazioni relative all'andamento dell'addestramento, come la training loss, le metriche di valutazione e altri parametri utili al monitoraggio del training;

- salva un checkpoint del modello;
- monitora le prestazioni del modello sul validation set al fine di valutarne la capacità di generalizzazione su dati non utilizzati per l'aggiornamento dei pesi.

Il processo di training termina al raggiungimento del numero di epoche prestabilito oppure anticipatamente tramite l'utilizzo di una callback di *Early Stopping*. Quest'ultima interrompe l'addestramento nel caso in cui la metrica monitorata non mostri miglioramenti per un determinato numero di epoche consecutive, contribuendo così a prevenire fenomeni di overfitting e a ridurre il costo computazionale complessivo. Grazie all'opzione `load_best_model_at_end = True`, al termine dell'addestramento viene automaticamente ripristinato il checkpoint del modello che ha ottenuto le migliori prestazioni rispetto alla metrica di valutazione monitorata sul validation set durante le fasi di evaluation.

Risultati

4.1 Risultati del training

L'analisi dell'andamento delle metriche registrate a ogni epoca durante il training consente di valutare il comportamento del modello sia in termini di apprendimento sia in termini di capacità di generalizzazione. In particolare, l'osservazione congiunta delle metriche calcolate sul training set e sul validation set permette di effettuare un confronto diretto tra le prestazioni sui dati utilizzati per l'addestramento e quelle ottenute su dati non osservati durante l'ottimizzazione.

Epoch	Training Loss (MSE)	Validation Loss (MSE)	MAE	R^2
1	0.0456	0.03133	0.13490	0.74751
2	0.0273	0.02924	0.12803	0.76434
3	0.0211	0.02796	0.12831	0.77469
4	0.0163	0.03133	0.13224	0.74748
5	0.0127	0.02960	0.12929	0.76148

Tabella 4.1: Andamento delle metriche di training e validazione per ogni epoca.

Dall’osservazione dei resoconti di training e validazione emerge come le migliori prestazioni vengano raggiunte già alla terza epoca, con un Validation Loss (MSE) pari a 0.0279, un Mean Absolute Error (MAE) di 0.1283 e un coefficiente di determinazione R^2 pari a circa 0.77. A partire dalla quarta epoca si registra un incremento del Validation Loss, nonostante il Training Loss continui a diminuire.

Tale divergenza rappresenta un chiaro segnale di overfitting: il modello tende progressivamente ad adattarsi alle specificità del dataset di addestramento, riducendo la propria capacità di generalizzare su dati non osservati. Questo comportamento risulta coerente con la dimensione del dataset utilizzato che, pur adeguata per un’operazione di fine-tuning (circa 15.000–17.500 recensioni dopo il bilanciamento), rimane relativamente contenuta rispetto alla complessità parametrica di un modello basato su architettura BERT.

L’adozione del meccanismo di *Early Stopping* ha consentito di mitigare ulteriormente tale fenomeno, interrompendo automaticamente il training dopo due epoche consecutive senza miglioramenti sulla metrica monitorata. Inoltre, l’opzione `load_best_model_at_end` ha garantito il ripristino del modello corrispondente alla migliore epoca di validazione, individuata nella terza, assicurando così l’utilizzo della configurazione più performante.

4.1.1 Valutazione andamento sul Validation Set

I valori ottenuti permettono una valutazione quantitativa delle prestazioni:

- Il Validation Loss (0.0279), definito come Mean Squared Error (MSE), indica che lo scarto quadratico medio tra predizione e valore reale è contenuto. Un valore prossimo allo zero suggerisce un’elevata prossimità tra output del modello e target effettivi.
- Il MAE (0.1283) rappresenta l’errore medio assoluto espresso sulla scala normalizzata [0,1]. Ciò implica che il modello commette, in media, un errore pari a circa il 13% dell’intervallo complessivo. In termini pratici, per un valore reale pari a 0.75, la predizione media si colloca approssimativamente nell’intervallo 0.62–0.88.

- Il coefficiente R^2 (0.77469) indica che il modello è in grado di spiegare circa il 77% della varianza osservata nei dati di validazione. Considerata la natura intrinsecamente soggettiva del sentiment e la complessità del linguaggio naturale, tale valore può essere considerato soddisfacente.

Nel complesso, dunque, i risultati evidenziano come il modello sia in grado di catturare in maniera efficace la relazione tra contenuto testuale e livello di criticità espresso, nonostante la disponibilità limitata di dati specifici per il dominio. La spiegazione di oltre tre quarti della variabilità del fenomeno, accompagnata da un errore medio contenuto, suggerisce una buona capacità predittiva.

Il lieve fenomeno di overfitting osservato dopo la terza epoca indica che un incremento del dataset in un successivo re-train potrebbe contribuire a migliorare ulteriormente le prestazioni e la robustezza del modello. Si può pertanto concludere che il sistema sviluppato risulta adeguato al task di regressione del sentiment, fornendo predizioni affidabili e costituendo una base solida per applicazioni operative di analisi automatica delle comunicazioni e-mail.

4.1.2 Valutazione andamento sul Test Set

In modo analogo a quanto effettuato per il validation set durante la fase di training, è stata condotta una valutazione finale sul test set, costituito da dati mai utilizzati né per l'addestramento né per la validazione del modello. Questa fase risulta fondamentale per stimare in maniera oggettiva la reale capacità di generalizzazione del sistema.

Metrica	Validation set	Test set
MSE	0.0279	0.0260
MAE	0.1283	0.1234
R^2	0.7700	0.7810

Tabella 4.2: Confronto tra le metriche di Validation e Test Set.

La coerenza tra le metriche risultanti dal validation set e quelle risultanti dal test set rappresenta un indicatore positivo di robustezza del modello. L'assenza di un deterioramento significativo delle prestazioni sul test set conferma che il sistema non si è limitato a memorizzare i dati di training, ma ha effettivamente appreso pattern linguistici trasferibili a nuovi esempi. Nel complesso, quindi, i risultati ottenuti dimostrano l'efficacia della strategia di fine-tuning adottata e attestano l'idoneità del modello nella predizione del sentiment.

4.2 Valutazione grafica

Per completare la valutazione delle prestazioni del modello, sono stati prodotti dei grafici relativi alle predizioni effettuate sia sul dataset di validazione sia sul dataset di test, al fine di rappresentare visivamente le performance del modello. In particolare, sono state considerate tre tipologie di rappresentazioni grafiche principali:

- Il grafico **Predicted vs True Values** mette in relazione i valori predetti dal modello con i valori reali (il sentiment reale). In condizioni ideali, i punti dovrebbero disporsi lungo la diagonale $y=x$, indicando una corrispondenza perfetta tra predizione e realtà. La dispersione dei punti attorno alla diagonale permette di valutare immediatamente l'accuratezza del modello e l'eventuale presenza di pattern sistematici di errore.

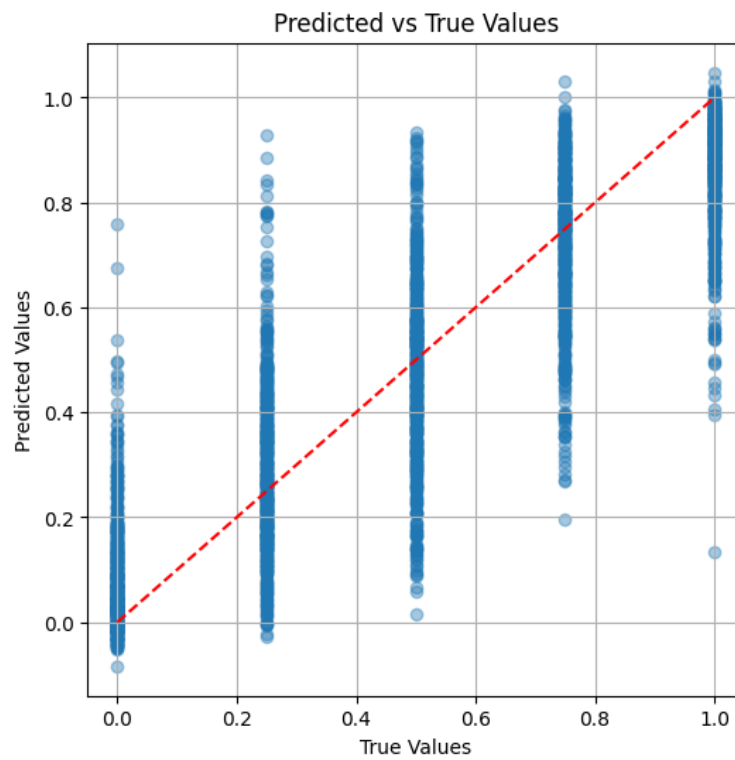


Figura 4.1: Grafico Predicted vs True Values sul Validation set

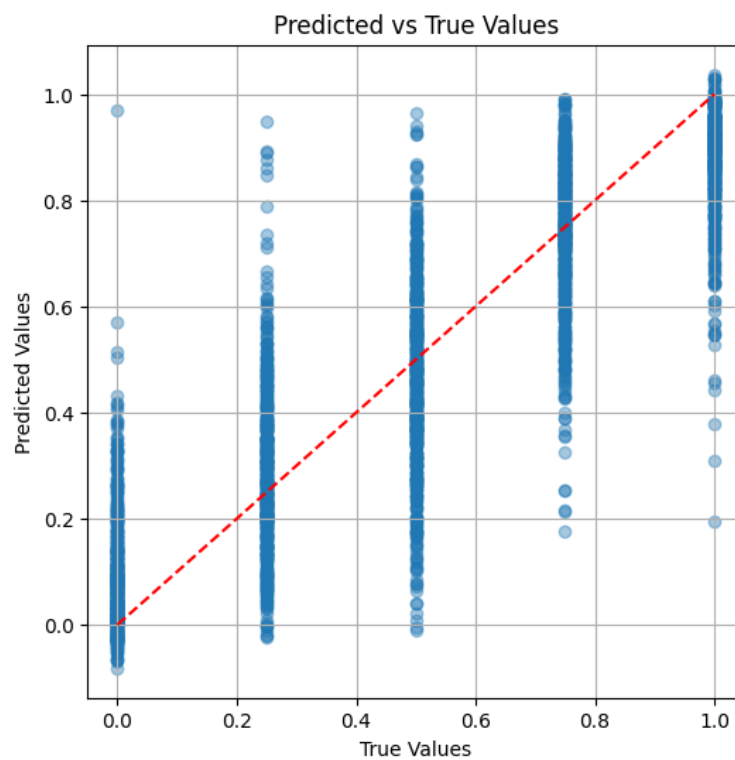


Figura 4.2: Grafico Predicted vs True Values sul Test set

Dall'analisi dei grafici si osserva che i punti tendono a disporsi lungo colonne verticali, fenomeno dovuto al mapping discreto dei rating in valori numerici continui distribuiti su cinque soglie. Tale struttura riflette la natura discreta del target originale. Tuttavia, la maggiore densità di punti si concentra in prossimità della linea diagonale di riferimento, evidenziando una buona accuratezza del modello, che produce predizioni generalmente vicine ai valori reali.

Si rileva una moderata dispersione, in particolare nelle classi intermedie (0.25–0.75), indicando una minore certezza del modello nella stima di recensioni caratterizzate da sentiment neutro o moderato rispetto agli estremi della scala.

- il grafico **Distribuzione degli errori** rappresenta la distribuzione dei residui (ovvero la differenza tra valore predetto e valore reale). Consente di osservare se gli errori sono centrati attorno allo zero, condizione desiderabile in un buon modello di regressione. Una distribuzione simmetrica e concentrata intorno allo zero indica l'assenza di bias sistematico; al contrario, una distribuzione sbilanciata suggerisce una tendenza del modello a sovrastimare o sottostimare i valori.

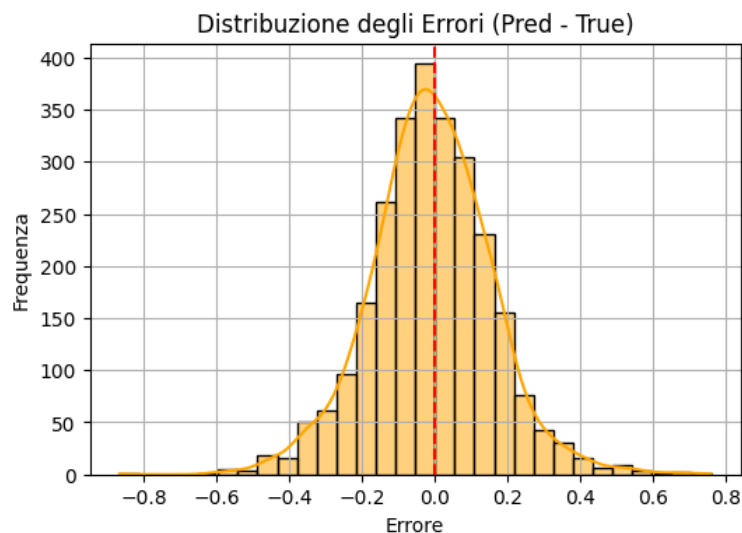


Figura 4.3: Grafico di Distribuzione degli errori sul Validation set

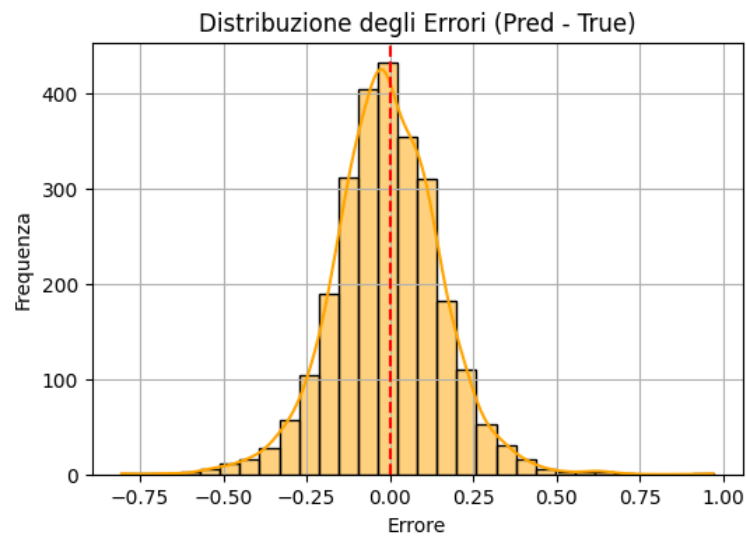
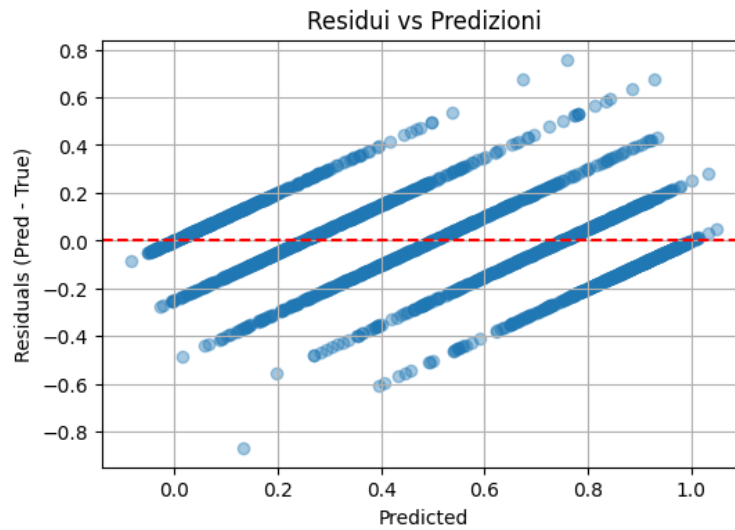
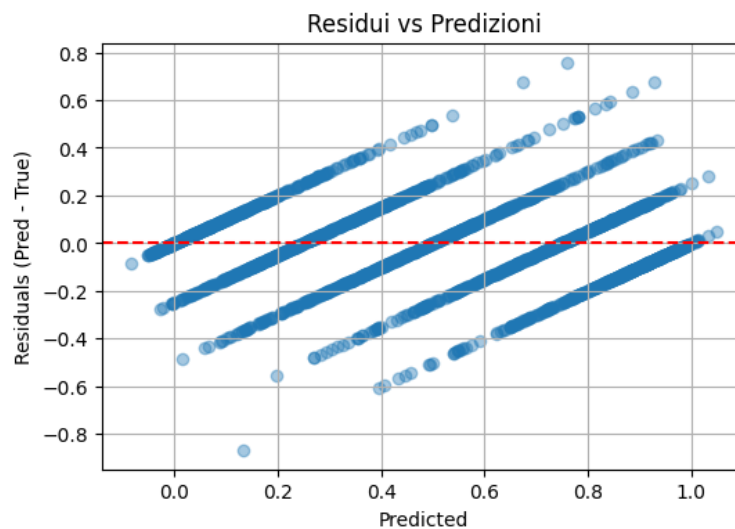


Figura 4.4: Grafico di Distribuzione degli errori sul Test set

La distribuzione degli errori risulta sostanzialmente simmetrica e centrata attorno allo zero, indicando l'assenza di un bias sistematico significativo nel modello. La forma approssimativamente gaussiana della distribuzione suggerisce buone proprietà statistiche dei residui, che appaiono distribuiti in modo regolare. Inoltre, la maggior parte degli errori è compresa nell'intervallo $[-0.15, +0.15]$, in coerenza con il valore di MAE ottenuto (trattato nella sezione 4.1), confermando che lo scostamento medio tra valori predetti e reali è contenuto.

- Il grafico **Residui vs Predizione** mette in relazione i residui con i valori predetti. L'obiettivo è verificare l'assenza di pattern strutturati. Idealmente, i punti dovrebbero essere distribuiti casualmente attorno alla linea dello zero. La presenza di forme riconoscibili (ad esempio strutture a imbuto o curve) potrebbe indicare problemi di eteroschedasticità o non linearità non catturata dal modello.

**Figura 4.5:** Grafico Residui vs Predizione sul Validation set**Figura 4.6:** Grafico Residui vs Predizione sul Test set

Dall'analisi della rappresentazione grafica si osserva che i residui risultano distribuiti in modo sostanzialmente simmetrico attorno allo zero, evidenziando l'assenza di un bias sistematico: il modello non manifesta una tendenza costante né alla sovrastima né alla sottostima dei valori reali.

Si rileva inoltre la presenza di bande diagonali, un fenomeno riconducibile alla natura discreta delle etichette originali. Sebbene il modello produca predizioni continue, queste tendono a concentrarsi in prossimità di tali soglie, comportamento coerente con la struttura discreta del target.

5.1 Comparazione con strumenti analoghi

In conclusione, è stata effettuata una comparazione tra il modello sviluppato e un Large Language Model general purpose basato sull'architettura LLaMA (Large Language Model Meta AI), impiegato per il medesimo task di analisi del sentiment. L'approccio basato su LLaMA opera secondo una logica differente rispetto al modello fine-tuned sviluppato nel presente progetto. In particolare, il testo dell'e-mail viene fornito direttamente in input al modello, il quale viene istruito tramite un prompt strutturato a predire il sentimento del messaggio, in modalità analoga all'interazione con sistemi conversazionali. L'approccio è dunque di tipo *prompt-driven*: il modello non viene ulteriormente addestrato sul dataset specifico del task, ma genera la classificazione basandosi esclusivamente sulle conoscenze linguistiche e semantiche acquisite durante la fase di pre-training su corpora di ampia scala.

1. Una prima differenza sostanziale riguarda quindi l'assenza di fine-tuning sul dominio specifico. Il modello produce una classificazione del sentiment (spesso accompagnata da una spiegazione testuale) facendo leva sulla propria capacità di generalizzazione.

Tuttavia, non essendo adattato direttamente al dataset di riferimento, la qualità della predizione risulta fortemente dipendente dalla formulazione del prompt e dalle caratteristiche linguistiche dell'e-mail analizzata.

2. Dal punto di vista dell'output, la predizione fornita da LLaMA è tipicamente di natura categorica (ad esempio positivo, negativo o neutro), senza la possibilità di ottenere una misura numerica continua del sentiment. Questo limita l'analisi granulare e l'integrazione con procedure statistiche più avanzate.
3. Un ulteriore aspetto riguarda le prestazioni computazionali. Essendo un LLM generico, LLaMA deve elaborare l'intero testo dell'e-mail e generare una risposta discorsiva completa, comprensiva di eventuale spiegazione della classificazione. Ciò comporta tempi di risposta generalmente superiori rispetto a modelli ottimizzati per task specifici, i quali producono direttamente l'output richiesto senza generazione testuale aggiuntiva.
4. Infine, assume particolare rilevanza la questione della sicurezza e della privacy dei dati. Qualora LLaMA venga utilizzato tramite infrastrutture esterne o servizi cloud, l'elaborazione di e-mail contenenti informazioni sensibili può comportare potenziali rischi legati alla protezione dei dati. L'invio di contenuti riservati a modelli non gestiti localmente richiede infatti specifiche garanzie in termini di conformità normativa e trattamento delle informazioni. Al contrario, un modello fine-tuned salvato e gestito internamente consente un pieno controllo sull'infrastruttura e sui flussi di dati, garantendo maggiori livelli di sicurezza.

5.1.1 Confronto empirico

Al fine di effettuare una comparazione omogenea tra i due modelli considerati, è stata introdotta una discretizzazione dell'output continuo prodotto dal modello sviluppato. In particolare, si è ritenuto opportuno associare al valore numerico predetto una classe discreta (negativo, neutro, positivo), così da rendere il confronto equo con approcci di natura classificativa come quello del modello basato su LLaMA.

Per determinare le soglie più appropriate, l'intero dataset ottenuto a seguito delle operazioni di pulizia e raffinamento è stato sottoposto al modello sviluppato. Le predizioni continue generate sono state successivamente analizzate mediante costruzione di matrici di confusione, valutando diverse possibili configurazioni di sogliatura.

L'analisi empirica ha evidenziato come la suddivisione dell'intervallo $[0, 1]$ secondo le soglie 0.4 e 0.6 consenta di ottenere la migliore separazione tra le tre classi, minimizzando le confusioni tra polarità opposte e rilevando qualche incertezza solo nella fascia intermedia. Pertanto, le classi sono state definite come segue:

Negativo: $y < 0.4$; Neutro: $0.4 \leq y < 0.6$; Positivo: $y \geq 0.6$.

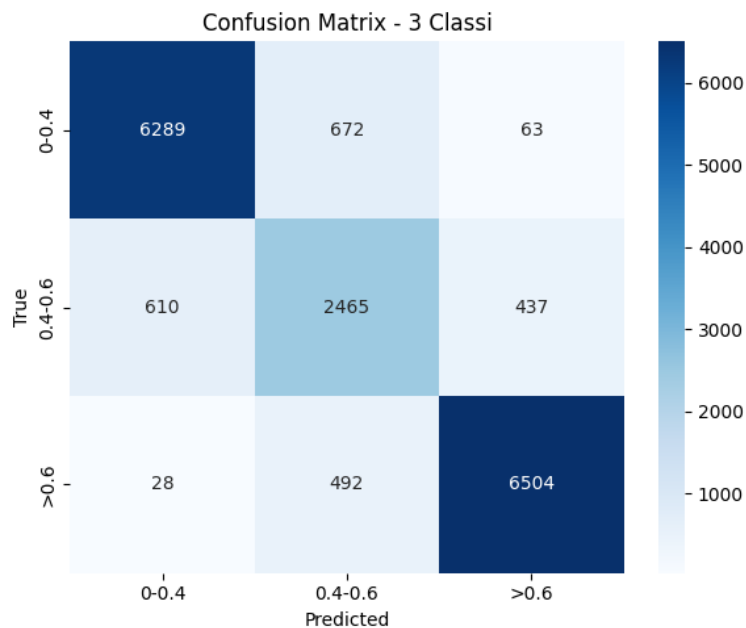


Figura 5.1: Matrice di confusione delle soglie scelte

Per la comparazione è stato costruito un corpus sperimentale composto da 25 e-mail redatte manualmente. Ciascun autore ha inoltre fornito una valutazione soggettiva del sentimento associato al proprio testo, utilizzata come riferimento qualitativo per il confronto.

Le e-mail sono state successivamente sottoposte a entrambi i modelli:

#	Modello LLaMA	Modello Progettato	Valutazione Umana
1	Positivo	Positivo (0.706313)	Positivo
2	Neutro	Negativo (0.378834)	Negativo
3	Positivo	Positivo (0.695130)	Positivo
4	Negativo	Negativo (0.178528)	Negativo
5	Positivo	Positivo (0.610413)	Positivo
6	Positivo	Positivo (0.753420)	Positivo
7	Negativo	Negativo (0.321840)	Negativo
8	Positivo	Positivo (0.653510)	Positivo
9	Negativo	Negativo (0.337634)	Negativo
10	Positivo	Medio (0.496337)	Positivo
11	Negativo	Negativo (0.169815)	Negativo
12	Positivo	Positivo (0.707988)	Positivo
13	Negativo	Negativo (0.356359)	Negativo
14	Positivo	Positivo (0.812338)	Positivo
15	Negativo	Negativo (0.229822)	Negativo
16	Positivo	Medio (0.502515)	Medio
17	Negativo	Medio (0.416985)	Medio
18	Positivo	Positivo (0.712428)	Positivo
19	Negativo	Negativo (0.188929)	Negativo
20	Positivo	Positivo (0.689351)	Positivo
21	Negativo	Medio (0.513048)	Medio
22	Negativo	Negativo (0.195180)	Negativo
23	Negativo	Medio (0.512196)	Medio
24	Negativo	Medio (0.479015)	Medio
25	Positivo	Medio (0.534046)	Medio

Tabella 5.1: Confronto tra modello LLaMA, modello progettato e valutazione umana

Dall'analisi comparativa emerge che il modello progettato presenta un tempo medio di inferenza significativamente inferiore (4.6 secondi complessivi contro 1.39 minuti del modello basato su LLaMA), risultando pertanto più adatto a scenari applicativi che richiedono rapidità di risposta.

Dal punto di vista qualitativo, il modello fine-tuned mostra una maggiore aderenza alla valutazione umana, con un solo caso di evidente misclassificazione sull'intero corpus analizzato. Il modello basato su LLaMA, pur fornendo risposte generalmente coerenti, tende invece a produrre classificazioni più dicotomiche (positivo/negativo), riducendo la capacità di cogliere le sfumature intermedie del sentimento.

Si può dunque concludere che la soluzione proposta rappresenti un'alternativa valida e tecnicamente più efficiente rispetto all'implementazione basata su LLaMA, migliorandone sia le prestazioni computazionali sia l'aderenza al giudizio umano nel contesto specifico del task considerato.

5.2 Possibili sviluppi futuri

Un'evoluzione particolarmente rilevante del sistema riguarda l'estensione dell'analisi agli allegati delle e-mail, con specifico riferimento ai documenti in formato PDF. Nell'ambito degli scambi e-mail, infatti, le informazioni di maggiore rilievo talvolta sono contenute proprio nei documenti allegati piuttosto che nel corpo del messaggio. L'integrazione dell'analisi documentale consentirebbe quindi di ottenere una valutazione più completa e affidabile del livello di criticità complessivo della comunicazione. Lo sviluppo di tale funzionalità può articolarsi nei seguenti passaggi.

1. **Riconoscimento del tipo di documento** La prima fase consiste nell'identificazione della natura del file allegato, distinguendo tra PDF nativamente testuali e PDF derivanti da scansione. Nel caso di PDF testuali, l'estrazione del contenuto può essere effettuata mediante librerie specializzate quali `pdfminer.six`, `PyPDF2` o `Apache Tika`, che consentono di recuperare il testo strutturato direttamente dal documento.

Qualora il PDF sia invece costituito da una scansione, risulta necessario applicare un livello intermedio di riconoscimento ottico dei caratteri (OCR). In tal caso possono essere impiegate soluzioni come Tesseract, docTR o servizi OCR in cloud, al fine di convertire l'immagine in testo elaborabile dal sistema.

2. **Segmentazione del documento** Una volta estratto il testo, è necessario procedere a una fase di segmentazione e pulizia strutturale. Il contenuto viene suddiviso in sezioni semanticamente rilevanti, eliminando elementi accessori quali intestazioni, piè di pagina, riferimenti automatici, metadati, informazioni di mittente/destinatario o altre porzioni non pertinenti all'analisi del contenuto sostanziale.

L'obiettivo è isolare il corpo informativo del documento, riducendo il rumore e migliorando la qualità dell'input fornito al modello di analisi del sentiment o della criticità.

3. **Analisi semantica e calcolo del punteggio** Successivamente, il testo viene sottoposto a una fase di pre-processing (rimozione di caratteri speciali, link, segni di punteggiatura ridondanti o altri elementi di disturbo) e quindi fornito al modello di regressione sviluppato. Il modello assegna un punteggio continuo di criticità, coerente con l'approccio già adottato per il corpo dell'e-mail.

Il punteggio derivato dal documento allegato viene poi combinato con quello relativo all'oggetto e al corpo del messaggio mediante una media ponderata, così da ottenere un valore finale rappresentativo dell'intera comunicazione.

5.3 Considerazioni finali

Il percorso progettuale sviluppato ha permesso di affrontare in modo strutturato un problema attuale e rilevante, coniugando l'analisi teorica dello stato dell'arte con l'implementazione concreta di una soluzione basata su tecniche di Intelligenza Artificiale. L'intero lavoro ha seguito una progressione coerente, dalla definizione degli obiettivi alla selezione del modello, fino alla fase di addestramento e valutazione sperimentale.

I risultati ottenuti evidenziano la validità dell'approccio adottato e confermano come l'utilizzo di modelli pre-addestrati, opportunamente adattati al dominio di interesse, possa rappresentare una strategia efficace per l'analisi automatica del testo. Il confronto ottenuto in fase di valutazione tra i valori ottenuti nei diversi set di dati, unitamente all'interpretazione dei grafici, ha consentito di verificare la coerenza dei risultati e di consolidare la solidità delle conclusioni raggiunte.

Al di là dei risultati tecnici, il progetto dimostra il potenziale delle tecnologie di Natural Language Processing nel supportare l'analisi strutturata di contenuti testuali complessi, contribuendo a rendere più efficiente l'elaborazione di grandi volumi di comunicazioni digitali.

Infine, il lavoro svolto non rappresenta un punto di arrivo, ma una base solida su cui costruire ulteriori sviluppi e approfondimenti, sia in termini di ampliamento funzionale sia di sperimentazione su scenari più ampi e articolati.

Bibliografia

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *NAACL-HLT*, 2019.
- [3] B. Pang and L. Lee, "Opinion mining and sentiment analysis," *Foundations and Trends in Information Retrieval*, 2008.
- [4] Y. Gong, G. Liu, Y. Xue, R. Li, and L. Meng, "A survey on dataset quality in machine learning," *Information and Software Technology*, 2023.
- [5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *ICLR*, 2019.
- [6] D. Montgomery, E. Peck, and G. Vining, *Introduction to Linear Regression Analysis*. Wiley, 2012.

Questa tesi ha contribuito a piantare un albero in Madagascar tramite il progetto Treedom.

<https://www.treedom.net/it/user/sesalab/event/sesa-random-forest>